EDUCATIONAL FULL-STACK PROJECT DOCUMENTATION

Zerodha Clone
Complete Project Guide

Architecture, trading engine, database design, API reference, Socket.IO, tournaments, and interview preparation based on the current repository — not a generic textbook.

Simulated paper trading · Not affiliated with Zerodha

Generated on: 11 October 2026
Source: backend/, dashboard/, frontend/ inspected from implementation
Backend tests run for this document: npm test → 160 passed, 0 failed

Table of contents

Chapter 1 — Project overview

Chapter 2 — Complete system architecture

Chapter 3 — Technology stack explained

Chapter 4 — Feature-by-feature documentation

Chapter 5 — Authentication and security

Chapter 6 — Database design

Chapter 7 — Trading engine

Chapter 8 — Tournament engine

Chapter 9 — API reference

Chapter 10 — Socket.IO and external integrations

Chapter 11 — Error handling and reliability

Chapter 12 — Testing and code quality

Chapter 13 — Setup and running

Chapter 14 — Limitations and future improvements

Chapter 15 — Technical interview preparation

Chapter 16 — Revision cheat sheet

Click a chapter title to jump (supported PDF viewers). Page numbers appear in the footer.

Chapter 1 — Project Overview

1.1 Executive Summary

Zerodha Clone is a production-grade, full-stack stock trading, portfolio management, and simulated investment platform modeled after India's premier discount broker, Zerodha (Kite). The system implements a full-featured financial trading engine supporting equity delivery (**CNC — Cash 'N Carry**) and intraday margin trading (**MIS — Margin Intraday Square-off**), market orders, limit orders with price triggers, cash and stock reservation locking, real-time market data streaming via WebSockets, competitive paper-trading tournaments, portfolio analytics, risk guard management, trading journals, strategy backtesting, and an integrated generative AI financial assistant powered by Google Gemini.

1.2 Core Project Goals

- **Institutional-Grade Trading Mechanics:** Replicate real stock market order execution mechanics, including available vs. used vs. reserved balances, limit order price condition triggers, and atomic inventory allocations.
- **Resilient Dual-Mode Execution:** Seamless operation with live external APIs (TrueData market feed, Twilio SMS OTP, MongoDB, Google Gemini) while maintaining offline and development fallbacks so the system remains interactive under all runtime environments.
- **Real-Time Data Distribution:** Event-driven WebSocket architecture utilizing Socket.IO for pushing real-time order status updates, price alert triggers, fund notifications, and tournament leaderboard rankings.
- **Gamified Simulated Learning:** Isolated tournament and paper-trading environments enabling risk-free trading competitions with independent virtual capital, real-time leaderboards, and automated prize/winner settlement.
- **Context-Aware Portfolio Intelligence:** Server-side AI assistant that dynamically queries user holdings, active positions, recent orders, and available funds to formulate personalized financial and risk commentary.

1.3 Complete Technology Stack

Layer	Technologies / Libraries	Key Architectural Role
Frontend Framework	React 19, Vite 8, React Router DOM 7	Modern Single Page Application (SPA) architecture, client-side routing, modular UI composition.
UI & Component Systems	Bootstrap 5.3, Material UI (@mui/material, @mui/icons-material), Emotion	Responsive trading terminal layouts, data tables, modals, drawer navigation, and Zerodha Kite UI aesthetics.
Data Visualization	Recharts, Custom Canvas Charts	Portfolio asset allocation pie charts, P&L performance timelines, historical candlestick/area stock charts.
Real-Time Client	Socket.IO Client 4.8	Subscribes to user private rooms (`user:{userId}`) and tournament competition rooms (`tournament:{id}`).
Backend Runtime	Node.js 22 (ES Modules), Express 5.2	High-concurrency asynchronous HTTP API server, request validation, authentication middleware, and business logic.
Database & ORM	MongoDB 7.0+, Mongoose 9.9	Document data persistence, schema validations, compound indexing, and multi-document ACID transactions.
Authentication & Security	JWT (jsonwebtoken 9.0), bcryptjs 3.0, Cookie-Parser, Passport.js, Joi	Stateless HTTP-only cookie JWT strategy, mobile OTP verification challenges, token version invalidation.

Layer	Technologies / Libraries	Key Architectural Role
External Integrations	@google/genai 2.27, TrueData SDK / WebSocket, Twilio Verify	Generative AI analysis (Gemini 2.5 Flash), live NSE/BSE tick feeds, SMS OTP dispatch.

1.4 Complete Repository Directory Structure

```
/
├── backend/                          # Express REST API, Mongoose models, and services
│   ├── config/
│   │   └── passport.js               # Passport local authentication strategy
│   ├── controller/                   # Request handlers & response formatters
│   │   ├── aiController.js           # AI Assistant endpoints & context synthesis
│   │   ├── authController.js         # Registration, login, profile, password management
│   │   ├── backtestController.js     # Strategy backtesting runner
│   │   ├── fundController.js         # Fund additions, withdrawals, ledger transactions
│   │   ├── holdingController.js      # CNC delivery stock holdings
│   │   ├── journalController.js      # Trading journal CRUD & tagging
│   │   ├── marketController.js       # Live quotes, historical OHLCV data
│   │   ├── notificationController.js # User alerts & read state updates
│   │   ├── orderController.js        # Market & limit order placement, cancellations
│   │   ├── otpAuthController.js      # Mobile OTP dispatch and verification
│   │   ├── positionController.js     # Intraday MIS positions & square-offs
│   │   ├── priceAlertController.js   # Target price alerts evaluation
│   │   ├── riskController.js         # Risk Guard limits & drawdown monitoring
│   │   ├── tournamentController.js   # Paper-trading competitions & leaderboards
│   │   ├── watchlistController.js    # Legacy watchlist controller
│   │   └── watchlistsController.js   # Multi-watchlist management & symbol ordering
│   ├── db/
│   │   └── connectDB.js              # Mongoose connection with timeout & offline resilience
│   ├── data/
│   │   ├── data.js                   # Seed & offline fallback data (holdings, positions, funds)
│   │   └── marketStocks.js           # NSE/BSE master ticker directory & instrument keys
│   ├── middleware/
│   │   ├── adminMiddleware.js        # Role-based access control (admin authorization)
│   │   ├── isLoggedInMiddleware.js   # JWT cookie extraction, signature verification, token
versioning
│   │   ├── orderValidation.js        # Joi schema validation for order placement
│   │   └── validateUser.js           # Joi registration/login payload validation
│   ├── models/                       # 15 Mongoose Schema Definitions
│   │   ├── BacktestHistoryModel.js   # Stored backtest runs & KPI metrics
│   │   ├── ChatModel.js              # AI assistant conversation threads & messages
│   │   ├── FundsModel.js             # Trading balance (available, used, reserved)
│   │   ├── FundTransactionModel.js   # Deposit & withdrawal ledger history
│   │   ├── HoldingsModel.js          # CNC equity delivery holdings
│   │   ├── JournalModel.js           # Trading journal entries with emotional/setup tags
│   │   ├── NotificationModel.js      # In-app notifications
│   │   ├── NotificationPreferenceModel.js# Granular user notification settings
│   │   ├── OrdersModel.js            # Master order records (Market/Limit, CNC/MIS)
│   │   ├── OtpChallengeModel.js      # Mobile OTP challenges with expiry & rate limiting
│   │   ├── PortfolioSnapshotModel.js # Historical equity snapshots for analytics
│   │   ├── PositionsModel.js         # Intraday MIS open/closed positions
│   │   ├── PriceAlertModel.js        # User price target alerts
│   │   ├── RiskSettingsModel.js      # Maximum daily loss, order sizing, and leverage caps
│   │   ├── TournamentModel.js        # Competition metadata, schedules, initial virtual capital
│   │   ├── TournamentOrderModel.js   # Virtual orders executed inside tournaments
│   │   ├── TournamentParticipationModel.js # Participant virtual balances, ROI, and rank
│   │   ├── UserModel.js              # User accounts, hashed passwords, client IDs
│   │   └── WatchlistListModel.js     # Multi-watchlist collections & symbols
│   ├── routes/                       # Express route definitions
│   └── services/                     # Business logic & external API wrappers
│       ├── aiService.js              # Google GenAI SDK interface & prompts
│       ├── backtestEngine.js         # Historical simulation execution engine
│       ├── checkPendingLimitOrders.js    # Limit order trigger evaluator on price ticks
│       ├── createOrderServices.js    # Atomic order execution & balance debit/credit
│       ├── limitOrderService/        # Limit execution & cancellation helpers
│       ├── tournamentService.js      # Tournament order matching & background lifecycle
│       └── trueDataService.js        # TrueData WebSocket & in-memory price cache
├── dashboard/                        # React trading platform (Kite interface)
├── frontend/                         # React public marketing portal (Zerodha home/pricing)
├── src/                              # Unified full-stack client application
```

```
├── docs/                              # Architectural specifications & technical documentation
├── index.html                         # Master HTML entry point
├── server.js                          # Full-stack server entry point (Express + Vite middleware)
└── package.json                       # Unified dependency and script manifest
```

Chapter 2 — Complete System Architecture

2.1 Overall System Architecture

The system utilizes a modern, unified full-stack architecture that cleanly separates client presentation from backend domain logic while enabling seamless single-origin execution without cross-origin resource sharing (CORS) friction.

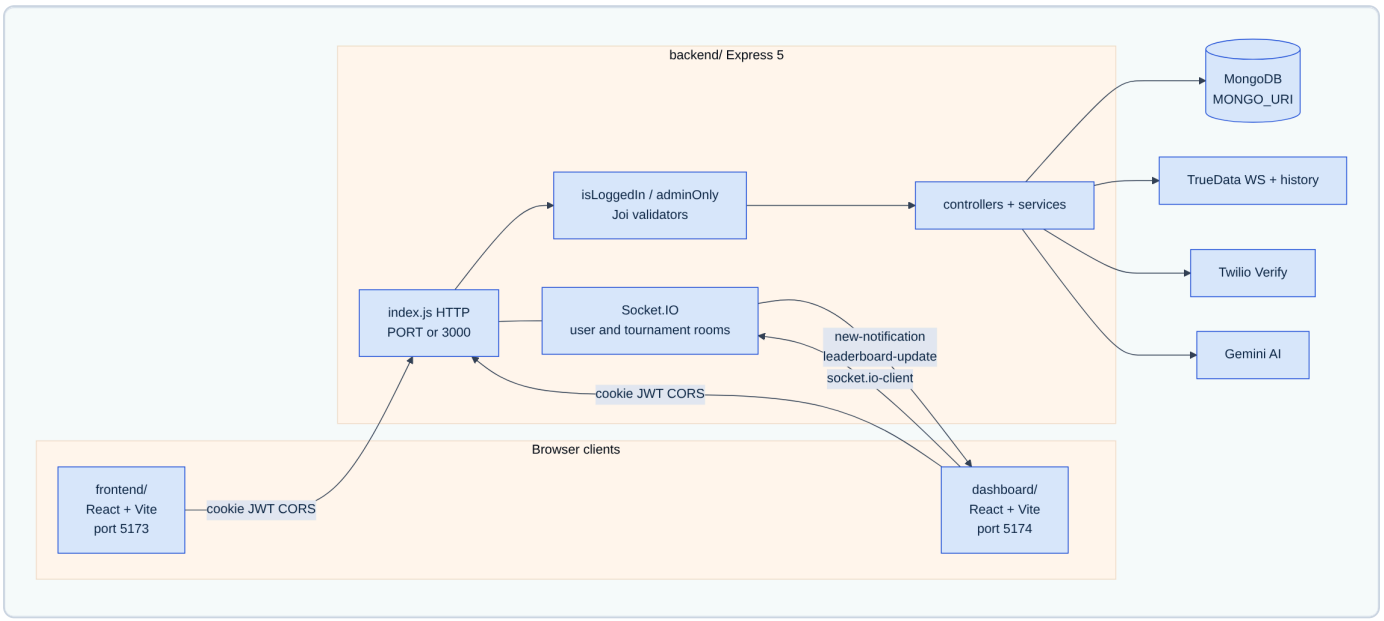


Figure 2.1: End-to-end component architecture, showing clients, Express layer, MongoDB, and external integrations.

Architectural Components Walkthrough:

1. **Browser Client Layer:** The presentation tier is built with React 19 and Vite. In the original repository, two distinct Vite apps existed: `frontend/` (marketing, landing pages, education) and `dashboard/` (Kite trading terminal). In AI Studio and production deployment, these are unified under a single origin on port 3000, eliminating browser cross-port redirects.
2. **Unified Express Gateway & Vite Middleware:** The Node.js 22 runtime server boots an Express 5 instance on port 3000. In development mode, Vite's internal development server is mounted as an Express middleware (`vite.middlewares`), handling HMR and JSX on-the-fly. In production, pre-compiled static assets in `dist/` are served directly with an SPA catch-all route.
3. **Middleware Pipeline:** Incoming HTTP requests pass through security and parsing middlewares:
    - `cors({ origin: true, credentials: true })`: Permits secure credentialed requests.
    - `cookieParser()`: Extracts the `token` cookie containing the signed JSON Web Token.
    - `isLoggedInMiddleware.js`: Decodes JWT, validates session active status, and checks `tokenVersion` to support global logout.
    - `orderValidation.js` / `validateUser.js`: Joi schema validation catching invalid inputs before controllers execute.
4. **Domain Services & State Machines:** Business logic is isolated from HTTP handlers into dedicated services (`createOrderServices.js`, `tournamentService.js`, `aiService.js`, `trueDataService.js`).
5. **Persistence Layer:** MongoDB stores structured documents across 15 schemas. When executing orders or balance modifications, Mongoose leverages native multi-document transactions with atomic session rollbacks.

6. **Real-Time Event Multiplexer (Socket.IO):** Attached to the native HTTP server instance, Socket.IO routes events into isolated rooms (`user:{userId}` and `tournament:{id}`).

2.2 HTTP Request Lifecycle

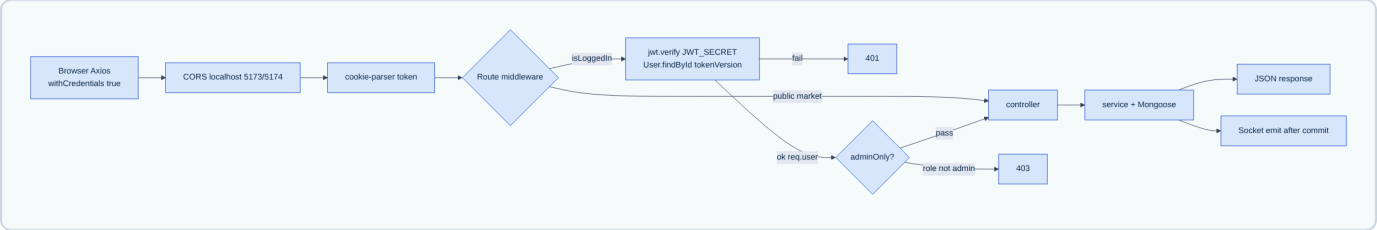


Figure 2.2: Detailed HTTP request lifecycle from browser interaction to database commit and socket dispatch.

When a user initiates an action in the UI (for example, clicking "Buy 10 Reliance CNC at ₹2,525"):

1. **HTTP Transmission:** The Axios client transmits a `POST /api/orders` request with `{ withCredentials: true }`, automatically attaching the browser's HTTP-only cookie.
2. **Middleware Verification:** Express matches the route. `isLoggedIn` extracts the cookie, runs `jwt.verify(token, JWT_SECRET)`, queries the user model to verify `tokenVersion`, and attaches `req.user = { userId, role, status }`.
3. **Payload Validation:** The Joi validation middleware parses `req.body` against `orderValidationSchema.js`. If validation fails (e.g., negative quantity or missing symbol), a 400 Bad Request is returned immediately without touching the database.
4. **Controller Execution:** `orderController.js` invokes `createOrderServices.js`, passing the sanitized payload and authenticated `userId`.
5. **Transactional Execution:** A MongoDB transaction session begins:
    - Calculates required capital: `qty * price`.
    - Checks `availableBalance >= requiredAmount`.
    - Updates user ledger: `availableBalance -= requiredAmount`, `usedBalance += requiredAmount`.
    - Creates/updates Holding or Position record.
    - Inserts Order record with status `completed`.
6. **Event Dispatch & Response:** Upon successful commit, a notification is written, Socket.IO emits `new-notification` to `user:{userId}`, and HTTP 201 Created is returned to the client. The frontend AppContext receives the response and triggers state re-fetching.

2.3 Real-Time WebSocket & Socket.IO Flow

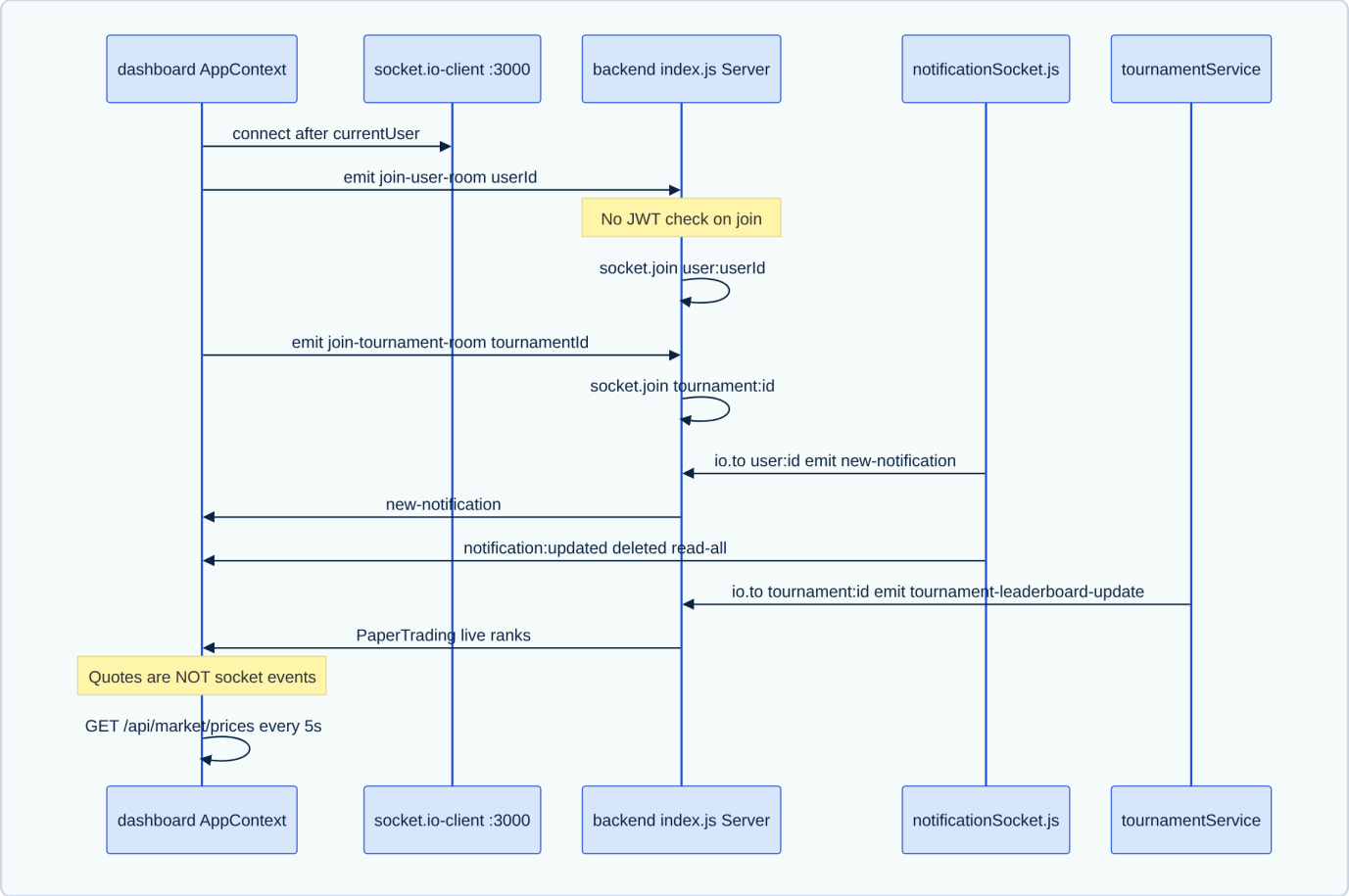


Figure 2.3: Socket.IO connection establishment, room joining, and server-to-client push events.

The real-time infrastructure provides sub-second push updates for:

- **Order Execution Notifications:** When limit orders trigger in the background via live market ticks, user alerts fire without polling.
- **Price Alerts:** Triggered when a stock reaches a user-defined price ceiling or floor.
- **Tournament Leaderboards:** Real-time rank and ROI calculations broadcast to all active competitors inside `tournament:{id}`.

Chapter 3 — Frontend Applications & Component Architecture

3.1 Overview of Frontend Structure

The presentation tier is designed as a modular, responsive Single Page Application (SPA) built with React 19, React Router DOM 7, and Vite 8. It delivers two distinct user experiences:

1. **Public Marketing Portal (Landing Application):** The institutional marketing website featuring the Zerodha brand homepage, transparent brokerage pricing calculators, ecosystem product showcases (Console, Coin, Kite Connect, Varsity), about pages, and mobile OTP signup/login interfaces.
2. **Trading Terminal (Kite Dashboard):** The secure, authenticated trading dashboard providing real-time market watchlists, buy/sell order execution modals, holdings breakdown, intraday positions, ledger balance management, portfolio performance analytics, risk guard monitoring, trading journals, strategy backtesting, paper-trading competitions, and an integrated AI assistant.

3.2 Core Component Hierarchy

```
App (AppProvider + BrowserRouter)
├── LandingLayout (Navbar + PageContent + Footer)
│   ├── HomePage (Hero, Stats, Pricing, Education, OpenAccount)
│   ├── AboutPage (Hero, Team, Philosophy)
│   ├── ProductPage (Kite, Console, Coin, Varsity, Universe)
│   ├── PricingPage (Brokerage charges, STT/CTT breakdown, Account charges)
│   ├── SupportPage (Ticket creation, Knowledge base)
│   ├── Login (Mobile OTP login + Demo account exploration)
│   └── SignUp (4-Step onboarding: Mobile OTP -> Registration -> Verification)
│
└── ProtectedRoute (Authentication guard & session verification)
    └── DashboardLayout
        ├── Sidebar (Collapsible navigation, active route highlight, theme toggle)
        ├── TopNavigation (Market summary indices, search bar, user avatar, notifications dropdown)
        ├── MarketWatchlist (Persistent left-panel ticker, multi-watchlists, bid/ask hover actions)
        │   ├── WatchListActions (Buy/Sell trigger buttons, depth chart view, pin action)
        │   ├── StockChartPopup (Interactive historical price canvas / candlestick charts)
        │   └── StockDetailsPopup (Fundamental metrics, 52-week high/low, P/E ratio, volumes)
        │
        ├── TradePopup (Floating Order Window: Buy/Sell, Market/Limit, CNC/MIS, margin calculator)
        ├── FundToast & OrderToast (Floating execution status notifications)
        │
        └── Main Page Content:
            ├── Dashboard (Portfolio summary cards, allocation doughnut, quick action triggers)
            ├── Holdings (CNC equity inventory, invested value, current value, total P&L, day P&L)
            ├── Positions (MIS intraday trades, buy/sell quantity, net profit, square-off trigger)
            ├── Orders (Order book: Pending Limit orders, Executed orders, Cancelled orders)
            ├── Funds (Available cash, Used margin, Reserved balance, Deposit & Withdraw modals)
            ├── PortfolioAnalytics (Drawdown charts, profit factor, win-loss ratio, sector breakdown)
            ├── RiskManagement (Risk Guard: Daily loss limit, maximum position size, leverage caps)
            ├── TradingJournal (Trade review logs, emotional tags, strategy reflections, setup notes)
            ├── StrategyBacktest (SMA Crossover simulation, profit factor, max drawdown metrics)
            ├── PaperTrading (Virtual tournament competitions, leaderboard ranks, virtual orders)
            ├── TournamentAdmin (Admin tournament creation, schedule rules, winner settlements)
            ├── AIAssistant (Conversational financial AI assistant with portfolio context injection)
            ├── Notifications (In-app alerts, price triggers, read/unread states, user preferences)
            └── Settings (Profile info, password updates, session revocation, theme selection)
```

3.3 Global State Management: AppContext.jsx

Application state is centralized within `AppContext.jsx` using React's Context API. This eliminates prop drilling across deeply nested components while maintaining reactivity across the trading terminal.

Key State Categories and Reactive Signals:

State Domain	State Variables	Purpose & Reactive Behavior
Authentication & User	`currentUser`, `userLoading`	Stores user profile (name, email, mobile, clientId, role). Polled upon mount via `GET /api/auth/me`.
Order Execution	`isTradePopupOpen`, `selectedStock`, `stockType`, `orderVersion`	Controls the trading window modal. Incrementing `orderVersion` triggers auto-refresh of order book and ledger.
Holdings & Positions	`holdings`, `isHoldingsLoading`, `holdingVersion`, `positionVersion`	Caches delivery holdings and intraday positions. Updated on order fulfillment or socket execution events.
Cash & Ledger	`availableBalance`, `usedBalance`, `reservedBalance`, `fundVersion`	Maintains the three-tier balance ledger. Automatically updated when orders are placed, triggered, or canceled.
Market Prices	`marketPrices`, `isMarketPricesLoading`	Stores latest tick data map (LTP, change, open, high, low, previous close) refreshed periodically and via WebSockets.
Notifications	`notifications`, `notificationPagination`	In-app notifications feed with real-time socket prepending and unread counter badges.
Visual Theme	`theme` ('light' | 'dark' | 'system')	Binds to `<html data-theme="...">` and synchronizes with `localStorage`.

3.4 TradePopup: The Trading Window

`TradePopup.jsx` implements the core order placement interface replicating Zerodha's Kite Order Window:

- **Color-Coded Modals:** Blue header for **BUY** orders, Orange header for **SELL** orders.
- **Product Selection:** Toggle between **CNC** (Cash 'N Carry delivery — 100% margin required) and **MIS** (Margin Intraday Square-off — leverage supported, auto square-off at 3:15 PM).
- **Order Types:**
  - **Market Order:** Executed immediately at current market LTP. Requires `availableBalance >= qty * LTP`.
  - **Limit Order:** User specifies a trigger limit price. Cash (for Buy) or stock shares (for Sell) are immediately placed into *reserved* status.
- **Dynamic Margin Calculator:** As the user adjusts the quantity and price inputs, the component dynamically calculates required funds: `Required Margin = Quantity * Price`, comparing it in real time against `availableBalance` and showing a warning badge if funds are insufficient.

Chapter 4 — Backend Architecture & Request Pipeline

4.1 Backend Architectural Pattern

The backend is engineered using Node.js 22 and Express 5.2 adhering to a layered **Controller-Service-Repository** architectural pattern. The codebase strictly separates HTTP transport concerns (status codes, headers, cookie parsing) from domain trading rules (balance math, limit order triggers, inventory locking) and persistence layers (MongoDB Mongoose models).

4.2 Middleware Pipeline Execution Order

Every incoming HTTP request traverses an ordered sequence of middlewares before reaching route controllers:

```
Incoming Request
      │
      ▼
1. cors({ origin: true, credentials: true })
   ├── Validates allowed origins
   └── Allows Authorization headers and cookies
      │
      ▼
2. bodyParser.json()
   └── Parses JSON request bodies into req.body
      │
      ▼
3. cookieParser()
   └── Extracts browser cookies into req.cookies (notably req.cookies.token)
      │
      ▼
4. passport.initialize()
   └── Sets up Passport authentication strategy framework
      │
      ▼
5. Route-Specific Middlewares:
   ├── validateUser / orderValidation (Joi schema validation on req.body)
   ├── isLoggedInMiddleware (JWT verification, session status, tokenVersion check)
   └── adminMiddleware (Verifies req.user.role === 'admin')
      │
      ▼
6. Domain Controller & Service
   └── Executes business logic, database transactions, socket emissions
      │
      ▼
7. Global Error Handler / Offline Fallback Middleware
   └── Catches uncaught errors, Mongoose connection drops, and network timeouts
```

4.3 Complete Backend Route Directory

Endpoint Prefix	Route File	Key Operations Supported
`/api/auth`	`backend/routes/auth.js`	Registration, password login, OTP mobile dispatch/verify, demo login, profile updates, session revocation, account deletion.
`/api/orders`	`backend/routes/order.js`	Placing market/limit orders (CNC/MIS), fetching user order book, canceling pending limit orders.
`/api/holdings`	`backend/routes/holding.js`	Fetching equity delivery holdings with live market valuation and P&L.

Endpoint Prefix	Route File	Key Operations Supported
`/api/positions`	`backend/routes/position.js`	Retrieving open/closed intraday MIS positions and square-off calculations.
`/api/funds`	`backend/routes/fund.js`	Retrieving available/used/reserved balances, depositing capital, withdrawing funds, transaction history ledger.
`/api/market`	`backend/routes/marketRoutes.js`	Retrieving single stock price, all stock quotes, historical OHLCV candles (1M, 1Y, 5Y, MAX).
`/api/watchlists`	`backend/routes/watchlistsRoutes.js`	Creating watchlists, renaming, reordering symbols, deleting watchlists.
`/api/price-alerts`	`backend/routes/priceAlertRoutes.js`	Setting target price alerts, listing active alerts, deleting price triggers.
`/api/notifications`	`backend/routes/notificationRoutes.js`	Fetching user notifications, marking individual/all notifications as read, updating notification preferences.
`/api/analytics`	`backend/routes/analyticsRoutes.js`	Portfolio performance history, win/loss ratios, sector allocations, max drawdown.
`/api/risk`	`backend/routes/riskRoutes.js`	Fetching and updating Risk Guard limits (daily loss caps, max leverage, position size limit).
`/api/journal`	`backend/routes/journalRoutes.js`	Creating trading journal entries, setup tags, psychological reflections, trade linkings.
`/api/backtest`	`backend/routes/backtestRoutes.js`	Running strategy backtests (e.g. SMA Crossover), retrieving simulation KPI metrics.
`/api/tournaments`	`backend/routes/tournamentRoutes.js`	Listing active tournaments, joining with virtual capital, placing virtual tournament orders, leaderboard stats, admin creation.
`/api/ai`	`backend/routes/aiRoutes.js`	Submitting chat prompts, injecting portfolio context, fetching chat history, deleting threads.

4.4 Controller vs. Service Decoupling

Rather than writing SQL/Mongo queries directly inside controller files, complex domain operations are delegated to service modules:

- `createOrderServices.js`: Coordinates validation, price retrieval, capital checks, and MongoDB transaction sessions.
- `checkPendingLimitOrders.js`: Invoked whenever fresh market ticks arrive; iterates over active pending limit orders to evaluate whether trigger prices are met.
- `tournamentService.js`: Manages tournament lifecycles (scheduled -> active -> completed), matches virtual limit orders against live market ticks, and recalculates leaderboards.
- `aiService.js`: Formulates prompt strings, enforces safety instructions, and handles communication with Google GenAI SDK.

Chapter 5 — Authentication, OTP & Security Architecture

5.1 Authentication Architecture Overview

Authentication is architected as a hybrid password and phone-based OTP system designed to mirror the multi-factor security expectations of modern Indian financial brokerages. The system uses signed **JSON Web Tokens (JWT)** transmitted via secure, **HTTP-only cookies**, paired with an active database-backed `tokenVersion` check that supports instant global session invalidation.

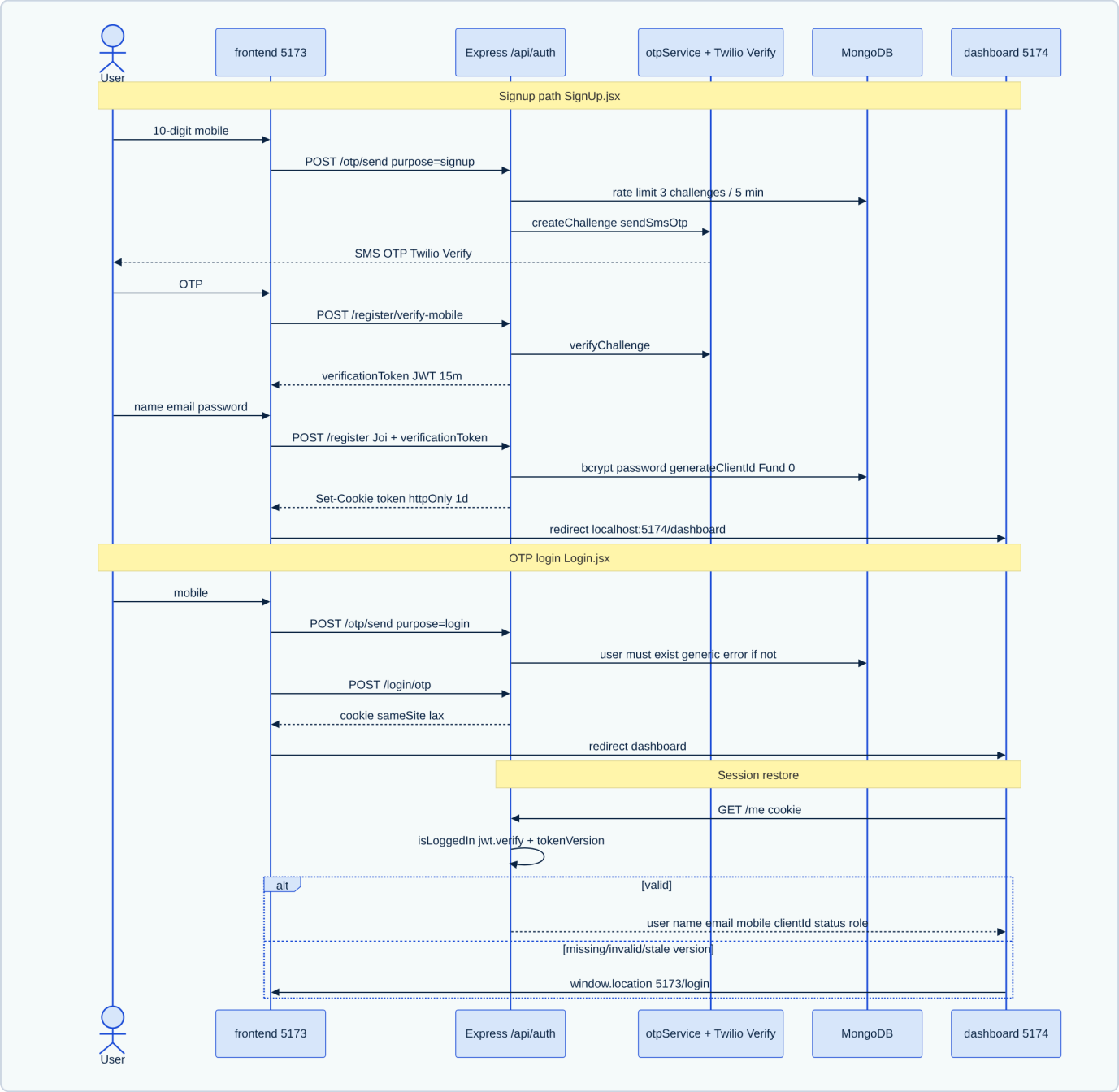


Figure 5.1: Authentication and OTP verification sequence diagram.

5.2 Mobile OTP Verification Challenge Flow

To prevent phone number spoofing and unauthorized account creation, mobile onboarding uses a multi-step challenge verification flow:

1. **Challenge Creation (`POST /api/auth/otp/send`):**
    - User submits a 10-digit mobile number.
    - System validates number using regex `/^\d{10}$/`.
    - Rate-limiting check: Queries `OtpChallengeModel` to ensure no more than 3 challenges were requested in the last 5 minutes. If exceeded, returns HTTP 429.
    - Existing unconsumed challenges for this number are invalidated (`isConsumed: true`).
    - Dispatches verification SMS via Twilio Verify. If Twilio credentials are not configured, the system logs a mock challenge code (`123456`) to ensure uninterrupted development and testing.
2. **Challenge Verification (`POST /api/auth/register/verify-mobile`):**
    - User submits the 6-digit OTP code received on SMS.
    - Twilio Verify checks code validity (or verifies against test mock code `123456`).
    - Upon approval, the challenge is marked `isConsumed: true`.
    - A short-lived signed JWT **verification token** is generated: `jwt.sign({ mobile, verified: true, challengeId }, JWT_SECRET, { expiresIn: '15m' })`.
    - This token must be included in the subsequent registration payload to prove phone ownership.

5.3 User Registration & Client ID Generation

Once verified, the user submits registration details (`POST /api/auth/register`):

- Validates name, unique email, verified mobile, password (minimum 6 characters), and `verificationToken`.
- Decodes the verification token, confirms that `decoded.mobile === req.body.mobile` and `decoded.verified === true`.
- Validates that the challenge record has not already been used for registration (`challenge.usedForRegistration === false`), then marks it used.
- Generates a unique Zerodha 6-character Client ID (e.g., `ZR9669`) using random alphanumeric character generation and collision detection.
- Hashes the password with `bcryptjs` using 10 salt rounds.
- Initializes a default `Funds` document for the new user with ₹1,00,000 initial virtual capital.
- Initializes a default `WatchlistList` titled "Watchlist 1" populated with blue-chip Indian equities (Reliance, TCS, Infosys, HDFC Bank, ICICI Bank, SBI, ITC, Bharti Airtel).
- Signs the session JWT and sets the HTTP-only cookie, automatically logging the user into their account.

5.4 Token Versioning & Global Session Revocation

A frequent security flaw in stateless JWT architectures is the inability to revoke tokens before their expiration window elapses. Zerodha Clone solves this using **Token Versioning**:

```
// JWT Payload structure:
{
  "userId": "68c123456789abcdef123455",
  "role": "trader",
  "tokenVersion": 3,
  "iat": 1791780813,
  "exp": 1792385613
}
```

When a user invokes `POST /api/auth/logout-other-sessions` or changes their password:

1. The backend executes `User.findByIdAndUpdate(userId, { $inc: { tokenVersion: 1 } })`.

2. The current active user is re-issued a fresh cookie with the incremented version.
3. Every subsequent request from any other device or browser containing the older `tokenVersion` fails the check in `isLoggedInMiddleware.js`:

```
if (user.tokenVersion !== decoded.tokenVersion) {
  return res.status(401).json({
    success: false,
    message: "Session expired. Please login again."
  });
}
```

4. All previous sessions are immediately invalidated across all devices without requiring a centralized token blacklist.

5.5 Cookie Security Specifications

Cookie Attribute	Configuration	Security Protection Provided
`httpOnly`	`true`	Prevents client-side JavaScript from accessing `document.cookie`, completely mitigating Cross-Site Scripting (XSS) token theft.
`sameSite`	`'lax'`	Protects against Cross-Site Request Forgery (CSRF) by preventing third-party origins from sending the authentication cookie during cross-site requests.
`maxAge`	`24 * 60 * 60 * 1000` (1 day)	Enforces session lifecycle boundaries, ensuring stale sessions expire naturally.
`secure`	Conditional (false in dev, true in HTTPS prod)	Ensures authentication cookies are transmitted exclusively over encrypted TLS connections in production.

Chapter 6 — Database Architecture & Schema Design

6.1 Database Architecture Overview

Persistence is managed by **MongoDB** using the **Mongoose** Object Data Modeling (ODM) framework. The schema design models high-frequency trading data, delivery equity holdings, multi-day ledger balances, audit trails, and virtual tournament portfolios.

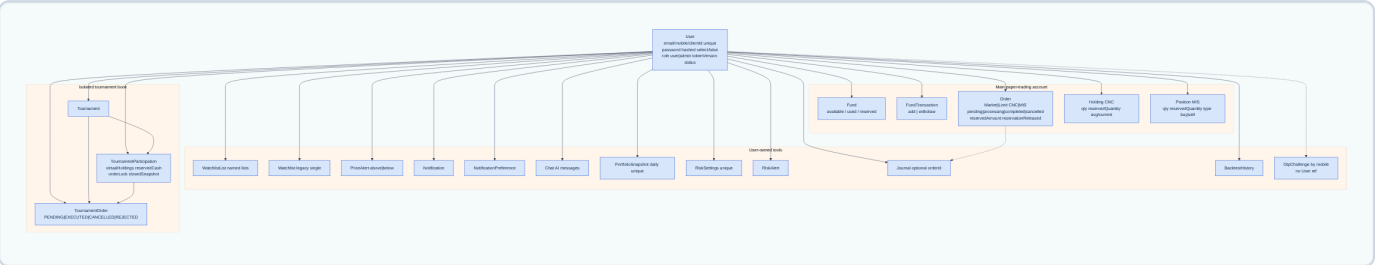


Figure 6.1: MongoDB Entity-Relationship diagram highlighting User references and operational collections.

6.2 Schema Specifications

1. User Model (`models/UserModel.js`)

Master account entity storing personal details, authentication credentials, and access control roles.

Field	Type	Constraints & Defaults	Description
`name`	String	Required, trim	User full legal name.
`email`	String	Required, unique, lowercase, trim	Account email address.
`mobile`	String	Required, unique, trim	10-digit verified Indian phone number.
`password`	String	Required, select: false	bcrypt-hashed password (excluded from default queries).
`clientId`	String	Required, unique, uppercase	6-character broker trading ID (e.g. `ZR9669`).
`role`	String	enum: ['trader', 'admin'], default: 'trader'	Access role for authorization.
`status`	String	enum: ['active', 'suspended'], default: 'active'	Account status.
`tokenVersion`	Number	default: 0	Session version counter for revocation.
`lastLogin`	Date	default: Date.now	Timestamp of last authenticated session.

2. Funds Model (`models/FundsModel.js`)

Maintains the three-tier balance ledger required for atomic order execution and margin management.

Field	Type	Constraints & Defaults	Description
`userId`	ObjectId / String	Required, ref: 'User', unique index	Associated trader ID.
`availableBalance`	Number	default: 100000, min: 0	Liquid capital available for new trades or withdrawal.
`usedBalance`	Number	default: 0	Capital currently deployed in active holdings or open positions.
`reservedBalance`	Number	default: 0, min: 0	Capital locked for pending buy limit orders.

3. Orders Model (`models/OrdersModel.js`)

Comprehensive order execution ledger tracking state transitions from pending to filled or canceled.

Field	Type	Constraints & Defaults	Description
`userId`	ObjectId / String	Required, ref: 'User', index	Trader placing the order.
`symbol`	String	Required, uppercase, trim	NSE/BSE ticker symbol (e.g. `RELIANCE`).
`qty`	Number	Required, min: 1	Number of shares to execute.
`price`	Number	Required, min: 0.05	Order execution price (or limit trigger price).
`mode`	String	enum: ['BUY', 'SELL']	Trade direction.
`orderType`	String	enum: ['MARKET', 'LIMIT'], default: 'MARKET'	Order trigger type.
`product`	String	enum: ['CNC', 'MIS'], default: 'CNC'	Equity delivery vs. Intraday margin.
`status`	String	enum: ['pending', 'processing', 'completed', 'cancelled']	Execution lifecycle state.
`reservedAmount`	Number	default: 0	Cash amount locked while in pending status.
`reservationReleased`	Boolean	default: false	Flag ensuring reserved funds are released exactly once.

4. Holdings Model (`models/HoldingsModel.js`)

Long-term equity inventory acquired through Cash 'N Carry (CNC) delivery purchases.

Field	Type	Constraints & Defaults	Description
`userId`	ObjectId / String	Required, index	Owner of the equity holding.
`symbol`	String	Required, uppercase	Equity ticker symbol.
`quantity`	Number	Required, min: 0	Unencumbered share quantity available to sell.
`averagePrice`	Number	Required	Volume-weighted average purchase price.
`reservedQuantity`	Number	default: 0	Shares temporarily locked in pending limit sell orders.
`product`	String	default: 'CNC'	Always CNC for delivery holdings.

5. Positions Model (`models/PositionsModel.js`)

Tracks leveraged Margin Intraday Square-off (MIS) trades that must be closed prior to market close.

Field	Type	Constraints & Defaults	Description
`userId`	ObjectId / String	Required, index	Trader holding the intraday position.
`symbol`	String	Required, uppercase	Ticker symbol.
`quantity`	Number	Required	Net active position share volume.
`averagePrice`	Number	Required	Entry price for intraday position.
`type`	String	enum: ['buy', 'sell']	Position direction (Long vs. Short).

Field	Type	Constraints & Defaults	Description
`product`	String	default: 'MIS'	Intraday product identifier.

6. Additional Specialized Models

- `FundTransactionModel`: Double-entry deposit and withdrawal ledger entries with unique timestamps.
- `TournamentModel`: Competition configurations (start time, end time, entry fee, initial virtual balance).
- `TournamentParticipationModel`: User tournament entries tracking virtual portfolio, rank, and ROI.
- `TournamentOrderModel`: Order records scoped specifically to tournament competition portfolios.
- `ChatModel`: Threaded AI conversation messages between trader and Gemini assistant.
- `WatchlistListModel`: User watchlists containing ordered arrays of tracked symbols.
- `PriceAlertModel`: Price threshold triggers (above/below condition checks).
- `RiskSettingsModel`: Risk Guard parameters (maximum daily loss, position limits, leverage constraints).
- `JournalModel`: Trade reflection notes with psychological tags (FOMO, Greed, Disciplined).
- `BacktestHistoryModel`: Stored strategy backtest results with calculated financial KPIs.

Chapter 7 — Trading Engine & Order Execution

7.1 Trading Engine Architecture Overview

The core of the platform is an institutional paper-trading engine that faithfully replicates the financial rules, balance management, and lifecycle state transitions of real-world stock exchanges. It enforces atomic transaction safety, prevents overdrafts, implements cash and inventory reservations, and matches limit orders in real time against live market ticks.

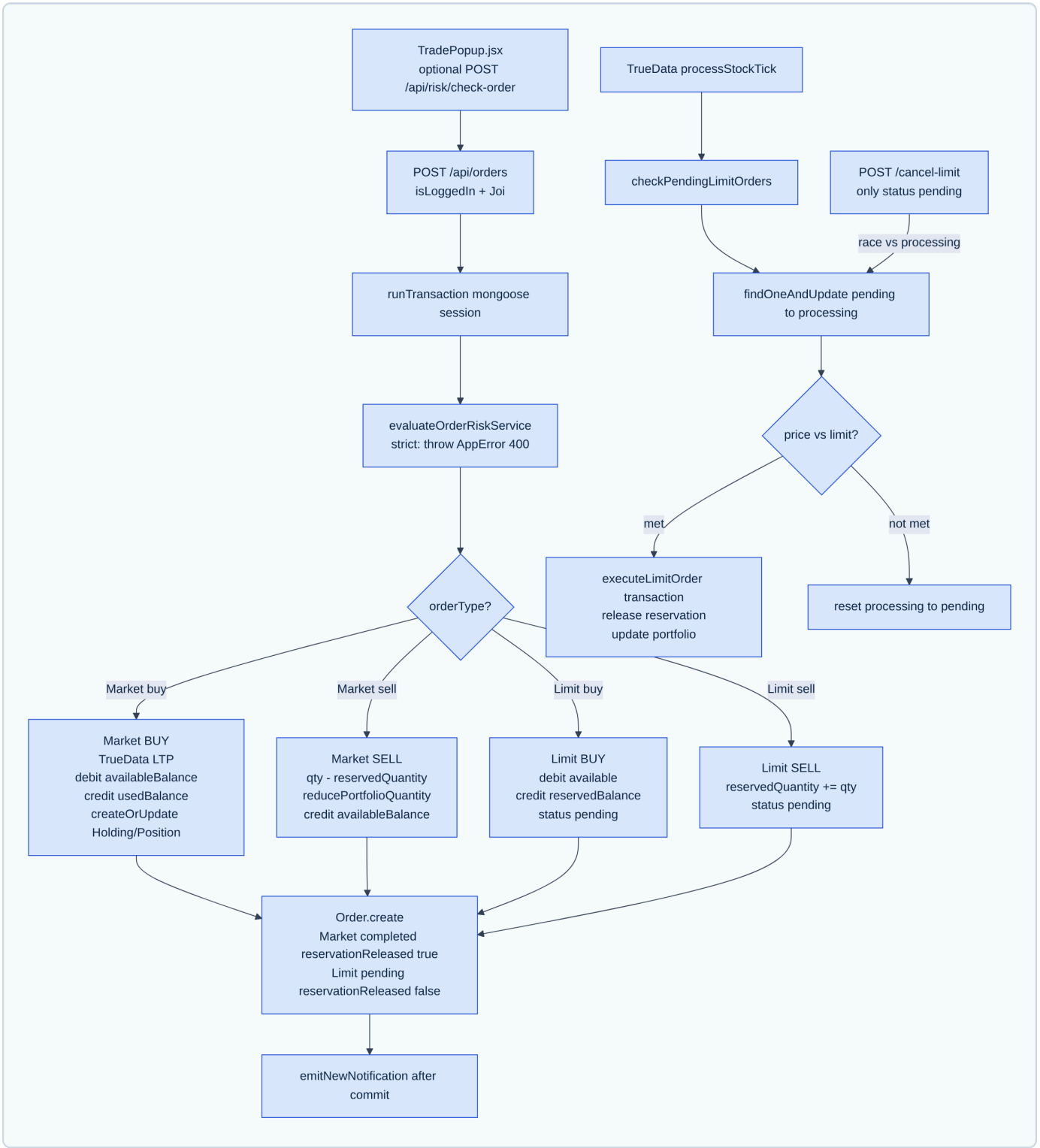


Figure 7.1: Comprehensive Market Order and Limit Order execution flow across BUY and SELL directions.

7.2 Market Order Execution (Immediate Fill)

When a trader submits a **Market Order**:

1. **Price Resolution:** The engine resolves the latest market price (LTP) from the in-memory cache populated by TrueData feeds.
2. **Pre-Trade Validation:**
   - **BUY Market:** The total required capital is calculated: `tradeAmount = qty * LTP`. The system queries `Funds` inside an atomic MongoDB session. If `availableBalance < tradeAmount`, the transaction aborts with an `Insufficient Funds` error.
   - **SELL Market (CNC):** The system queries `Holding` for the user and symbol. If `holding.quantity < qty`, the transaction aborts with an `Insufficient Quantity` error. Short selling is prohibited in CNC delivery.
3. **Atomic Ledger & Inventory Mutation:** Inside a MongoDB transaction session:
   - **BUY CNC:**

```
Funds.updateOne({ userId }, {
  $inc: { availableBalance: -tradeAmount, usedBalance: tradeAmount }
});
Holding.findOneAndUpdate({ userId, symbol }, {
  $inc: { quantity: qty },
  $set: { averagePrice: newWeightedAverage, currentPrice: LTP }
}, { upsert: true });
```

   - **SELL CNC:**

```
Funds.updateOne({ userId }, {
  $inc: { availableBalance: +tradeAmount, usedBalance: -originalCostBasis }
});
Holding.updateOne({ userId, symbol }, {
  $inc: { quantity: -qty }
});
```

     If the remaining holding quantity reaches 0, the holding document is removed.
4. **Order Insertion:** An order document is created with `status: 'completed'`, `orderType: 'MARKET'`, and `reservationReleased: true`.

7.3 Limit Order Lifecycle & Reservation Engine

A **Limit Order** allows a trader to specify an exact maximum price they are willing to pay (Limit Buy) or minimum price they are willing to accept (Limit Sell). Because execution may occur minutes or hours later, the engine locks capital or inventory to guarantee settlement when the market condition triggers.

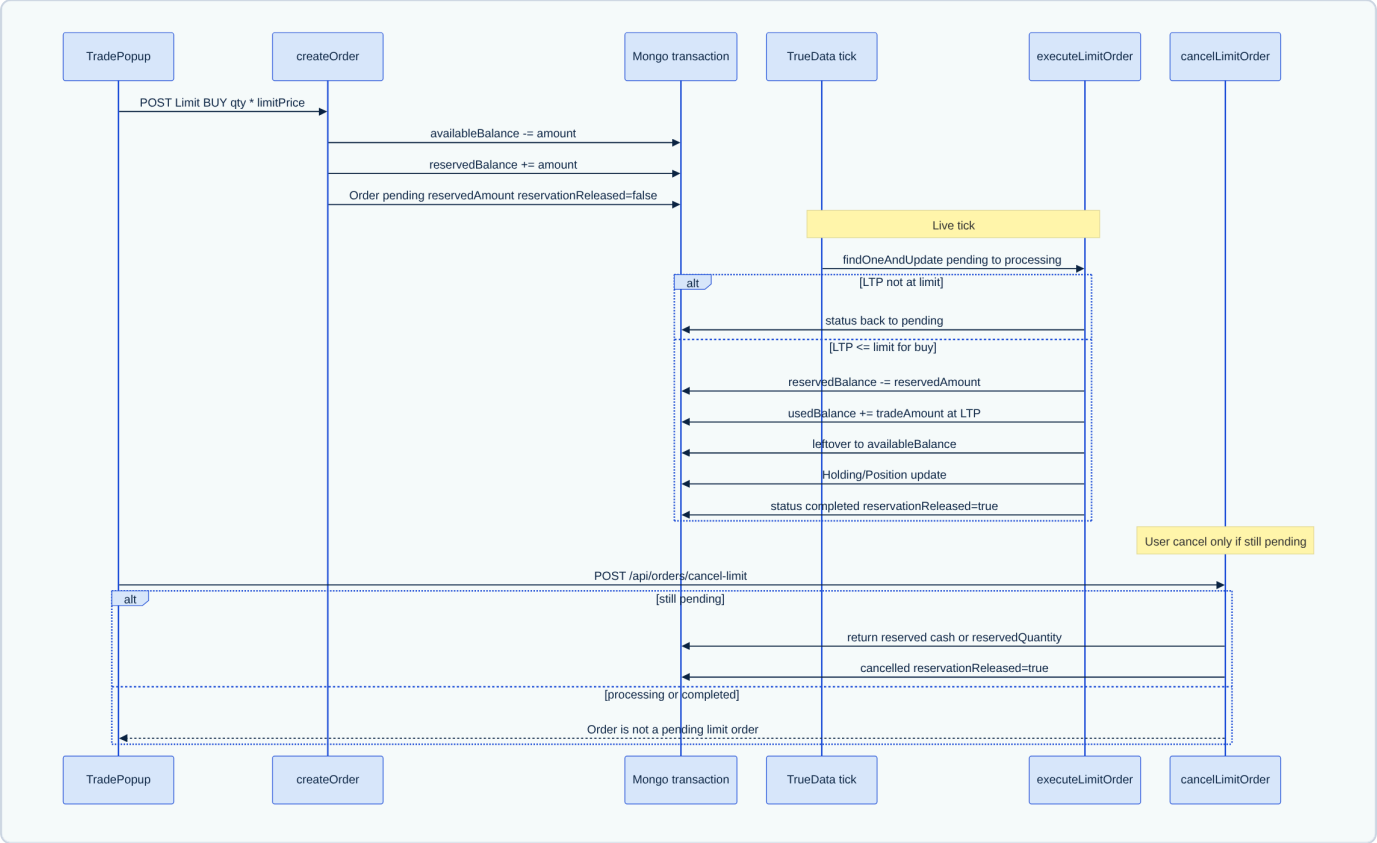


Figure 7.2: Limit order locking, background tick matching, execution adjustment, and user cancellation.

Step-by-Step Limit Order Lifecycle:

1. **Placement & Reservation (`createOrderServices.js`):**
    - **Limit BUY:** The maximum required cash is calculated: `reservedAmount = qty * limitPrice`. This amount is moved immediately from `availableBalance` to `reservedBalance`:

      ```
      Funds.updateOne({ userId }, {
        $inc: { availableBalance: -reservedAmount, reservedBalance: +reservedAmount }
      });
      ```

      The order is recorded with `status: 'pending'`, `reservedAmount`, and `reservationReleased: false`.
    - **Limit SELL:** The share quantity is reserved from the holding:

      ```
      Holding.updateOne({ userId, symbol }, {
        $inc: { quantity: -qty, reservedQuantity: +qty }
      });
      ```

2. **Background Tick Evaluation (`checkPendingLimitOrders.js`):** Whenever TrueData streams a price update (e.g., `RELIANCE LTP = ₹2,480`):
    - The worker queries for active pending orders for that symbol: `Order.find({ symbol, status: 'pending' })`.
    - **Condition Check:**
        - **BUY Limit:** Triggers if `currentLTP <= order.price`.
        - **SELL Limit:** Triggers if `currentLTP >= order.price`.

- **Two-Phase Execution Lock:** To prevent race conditions from concurrent ticks, the worker uses an atomic lock:

```
const lockedOrder = await Order.findOneAndUpdate(
  { _id: order._id, status: 'pending' },
  { $set: { status: 'processing' } },
  { new: true }
);
```

If another thread already acquired the lock, `lockedOrder` is null and execution gracefully skips.

3. **Execution Settlement & Cash Refund:** Inside a transaction session:
   - The actual cost at the market execution price is computed: `actualCost = qty * currentLTP`.
   - Because `currentLTP <= limitPrice`, the actual cost is typically less than or equal to the reserved cash!
   - The leftover savings are returned directly to the user's available balance:

```
const refundAmount = order.reservedAmount - actualCost;
Funds.updateOne({ userId }, {
  $inc: {
    reservedBalance: -order.reservedAmount,
    usedBalance: +actualCost,
    availableBalance: +refundAmount // Leftover savings returned!
  }
});
```

   - Holding is credited with shares: `quantity += qty`.
   - Order is updated to `status: 'completed'`, `reservationReleased: true`.
4. **User Cancellation (`cancelLimitOrderService.js`):** If the trader cancels a pending limit order before it triggers:
   - Verifies order is in `status: 'pending'` and `reservationReleased: false`.
   - Atomically unlocks the reserved cash or reserved shares back to available status.
   - Marks order `status: 'cancelled'`, `reservationReleased: true`.

7.4 Mathematical Worked Example: Limit Buy Execution

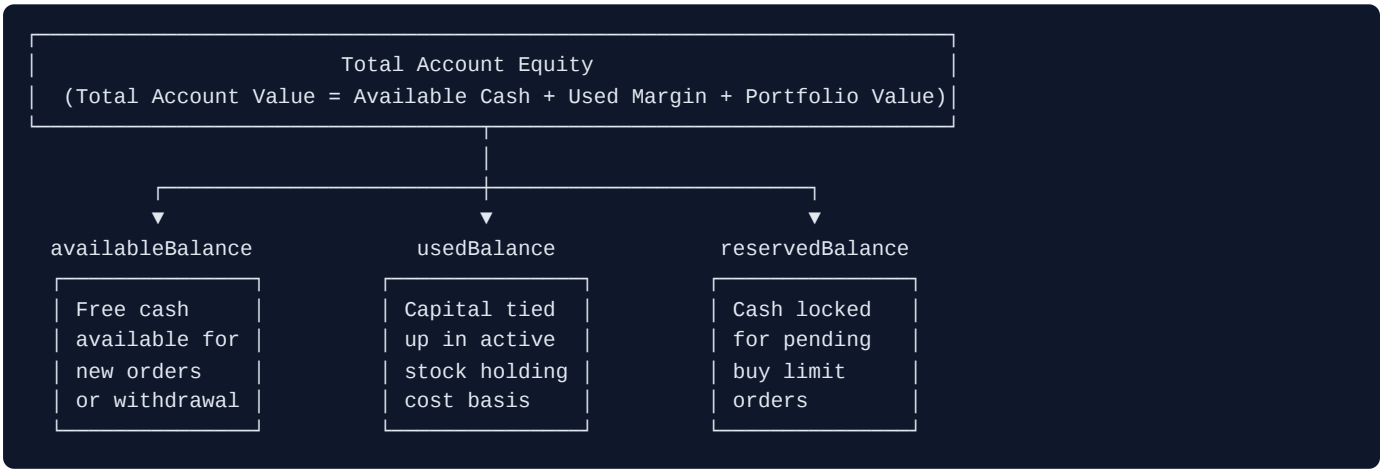
> **Scenario:**
>
> - User has ₹1,00,000 available balance.
> - Places a Limit Buy for 10 shares of Reliance at Limit Price ₹2,500.
> - Market dips to ₹2,480, triggering execution.
>
> **Step 1: Placement**
>
> - `reservedAmount = 10 * ₹2,500 = ₹25,000`
> - `availableBalance = ₹1,00,000 - ₹25,000 = ₹75,000`
> - `reservedBalance = ₹25,000`
> - `usedBalance = ₹0`
>
> **Step 2: Execution at ₹2,480**
>
> - `actualCost = 10 * ₹2,480 = ₹24,800`
> - `refundAmount = ₹25,000 - ₹24,800 = ₹200`

- `reservedBalance = ₹25,000 - ₹25,000 = ₹0`
- `usedBalance = ₹0 + ₹24,800 = ₹24,800`
- `availableBalance = ₹75,000 + ₹200 = ₹75,200`
- `Total Account Equity = ₹75,200 (cash) + ₹24,800 (stock value) = ₹1,00,000`

Chapter 8 — Funds Management & Ledger Architecture

8.1 The Three-Tier Balance Model

Standard e-commerce architectures store a single monolithic wallet balance (e.g. `wallet = 500`). In financial trading systems, a single balance is inadequate because active holdings, margin exposure, and unfilled limit orders require separate capital allocations. Zerodha Clone implements an institutional **three-tier balance ledger**:



8.2 Fund Balance Invariants & Math Equations

The system enforces strict invariants on every transaction:

1. **Non-Negativity Constraint:** `availableBalance >= 0` and `reservedBalance >= 0` at all times. Database schema enforces `min: 0`.
2. **Limit Order Placement Invariant:**

   `availableBalance_new = availableBalance_old - (qty * limitPrice)`

   `reservedBalance_new = reservedBalance_old + (qty * limitPrice)`
3. **Limit Order Execution Invariant:**

   `reservedBalance_new = reservedBalance_old - reservedAmount`

   `usedBalance_new = usedBalance_old + (qty * LTP)`

   `availableBalance_new = availableBalance_old + (reservedAmount - (qty * LTP))`
4. **Sell Holding Fulfillment Invariant:**

   `usedBalance_new = usedBalance_old - (qty * averagePurchasePrice)`

   `availableBalance_new = availableBalance_old + (qty * executionPrice)`

   (Net Realized Profit/Loss immediately reflects in the increase/decrease of available cash!)

8.3 Deposits, Withdrawals & Transaction Ledger

All capital deposits and withdrawals are managed via `backend/controller/fundController.js`:

- **Adding Funds (`POST /api/funds/add`):**

  Traders deposit virtual capital (e.g. ₹50,000 via simulated UPI or Netbanking). The transaction atomically increments `availableBalance` and creates a record in `FundTransactionModel` with `type: 'deposit'`, `status: 'completed'`.

- **Withdrawing Funds (`POST /api/funds/withdraw`):**

  Traders withdraw capital back to their bank account. The system validates that `amount <= availableBalance`. Funds deployed in active stocks (`usedBalance`) or reserved in open limit orders (`reservedBalance`) cannot be withdrawn until sold or canceled.

8.4 Transaction Helper: Atomic MongoDB Sessions

To guarantee complete ACID compliance without partial state corruption, database operations that span multiple collections (e.g. debiting funds and creating holdings) utilize `backend/helpers/transactionHelper.js`:

```
export const runTransaction = async (callback) => {
  const session = await mongoose.startSession();
  session.startTransaction();
  try {
    const result = await callback(session);
    await session.commitTransaction();
    return result;
  } catch (error) {
    await session.abortTransaction();
    throw error;
  } finally {
    session.endSession();
  }
};
```

If any step inside `callback` throws an error (e.g., duplicate key, network blip, or constraint violation), the entire sequence aborts, rolling back any partial modifications to funds, orders, or holdings.

Chapter 9 — Holdings, Positions & P&L Calculations

9.1 CNC Delivery Holdings vs. MIS Intraday Positions

Following Indian market structure, the platform distinguishes between two trading products:

Dimension	Cash 'N Carry (CNC) — Holdings	Margin Intraday Square-off (MIS) — Positions
Investment Horizon	Multi-day to multi-year long-term investment.	Single trading session (closed before 3:15 PM IST).
Leverage Allowed	1x (No leverage: 100% upfront cash required).	5x intraday leverage supported.
Short Selling	Not permitted (must own shares before selling).	Permitted (sell high, buy back lower before close).
Storage Collection	`holdings` collection (HoldingsModel).	`positions` collection (PositionsModel).

9.2 Volume-Weighted Average Purchase Price Formula

When a trader purchases additional shares of an already owned stock at a different market price, the new cost basis is calculated using the **Volume-Weighted Average Price** formula:

```
New Average Price = ( (Existing Qty * Existing Avg Price) + (New Qty * Purchase Price) ) / ( Existing Qty + New Qty )
```

Worked Example:

> - Trader currently owns **10 shares of TCS** with an average cost of **₹3,200** (Invested = ₹32,000).
> - Trader buys **5 additional shares of TCS** at the current market price of **₹3,500** (New Outlay = ₹17,500).
>
> `New Total Quantity = 10 + 5 = 15 shares`
>
> `New Total Invested = ₹32,000 + ₹17,500 = ₹49,500`
>
> `New Average Price = ₹49,500 / 15 = ₹3,300 per share`

9.3 Real-Time P&L Equations

The frontend Kite dashboard dynamically calculates real-time profit and loss metrics across every holding and for the aggregate portfolio:

1. Individual Holding Total P&L

```
Holding Invested Value = Quantity * Average Price
Holding Current Value  = Quantity * Current Market Price (LTP)
Total Profit / Loss    = Holding Current Value - Holding Invested Value
Percentage Return (%)  = ( Total Profit / Loss / Holding Invested Value ) * 100
```

2. Individual Holding Day P&L

Measures the profit or loss generated exclusively within the current trading session:

```
Day Profit / Loss   = Quantity * ( Current Market Price - Previous Day Close Price )
Day Return (%)      = ( ( Current Market Price - Previous Close ) / Previous Close ) * 100
```

3. Portfolio Aggregate Metrics (PortfolioSummary.jsx)

- `Total Portfolio Invested = Sum of all (Quantity * Average Price)`

- `Total Portfolio Current = Sum of all (Quantity * LTP)`
- `Net Unrealized P&L = Total Portfolio Current - Total Portfolio Invested`
- `Net Day P&L = Sum of all (Quantity * (LTP - Previous Close))`

9.4 Sell Inventory Reservation (Anti-Double-Spend)

To prevent traders from placing multiple sell orders exceeding their actual owned shares:

1. When a trader places a Limit Sell for 4 shares out of 10 owned shares:

```
Holding.updateOne({ userId, symbol }, {
  $inc: { quantity: -4, reservedQuantity: +4 }
});
```

2. The available quantity immediately drops to 6 shares. If the user attempts to place another sell order for 7 shares, the system validates against `quantity` (6) and rejects the request.
3. When the limit order executes, `reservedQuantity` is decremented by 4.
4. If the user cancels the order, `quantity` is incremented by 4 and `reservedQuantity` is decremented by 4.

Chapter 10 — Market Data & TrueData Integration

10.1 Market Data Ingestion Overview

Real-time market price feeds are ingested via **TrueData**, an authorized data vendor for the National Stock Exchange of India (NSE) and Bombay Stock Exchange (BSE). The platform consumes real-time tick streaming over persistent WebSocket connections and historical candle records via HTTP REST endpoints.

10.2 TrueData WebSocket Client (trueDataService.js)

The native WebSocket client manages authentication, subscription, and parsing of binary/JSON touchline ticks:

```
const url = `wss://push.truedata.in:${port}?user=${username}&password=${password}`;
wsConnection = new WebSocket(url);

wsConnection.on('message', (data) => {
  const jsonObj = JSON.parse(data);
  if (jsonObj.success && jsonObj.message === 'TrueData Real Time Data Service') {
    // Authenticated! Subscribe to 14 core NSE blue-chips:
    wsConnection.send(JSON.stringify({
      method: 'addsymbol',
      symbols: ['RELIANCE', 'TCS', 'INFY', 'HDFCBANK', 'ICICIBANK', 'SBIN', 'ITC', ...]
    }));
  } else if (jsonObj.symbollist) {
    // Process touchline tick records
    jsonObj.symbollist.forEach(processStockTick);
  }
});
```

Symbol Normalization:

To prevent mismatch bugs between exchanges and user inputs, all symbols pass through `normalizeSymbol(symbol)`:

```
const normalizeSymbol = (symbol) =>
  String(symbol || "")
    .trim()
    .toUpperCase()
    .replace(/^NSE:/i, "")
    .replace(/^BSE:/i, "");
```

10.3 Touchline Tick Parser & In-Memory Cache

Each incoming tick is transformed into a standardized price object and indexed into a fast in-memory `Map`:

TrueData Field Index	Normalized Attribute	Sample Value	Description
`item[0]`	`symbol`	`"RELIANCE"`	Ticker identifier.
`item[3]`	`currentPrice` (LTP)	`2525.50`	Last Traded Price.
`item[10]`	`previousClose`	`2490.00`	Official previous day closing price.
`Computed`	`priceChange`	`+35.50`	`LTP - previousClose`.
`Computed`	`priceChangePercent`	`+1.43%`	`(priceChange / previousClose) * 100`.
`item[7]`	`open`	`2495.00`	Session open price.

TrueData Field Index	Normalized Attribute	Sample Value	Description
`item[8]`	`high`	`2535.00`	Session high watermark.
`item[9]`	`low`	`2480.00`	Session low watermark.
`item[6]`	`volume`	`1,420,000`	Total traded share volume for session.

10.4 Offline & Development Resilient Fallback Cache

In development environments, sandboxes, or during off-market hours (weekends and evenings), external WebSocket push feeds may be unavailable. If unhandled, empty price maps cause trading calculations, order evaluation, and watchlist rendering to crash.

Zerodha Clone solves this by initializing `latestTrueDataPrices` with a high-fidelity static baseline cache covering all 14 supported symbols. When live TrueData feeds connect, the live ticks overwrite the baseline cache in real time. If live connectivity is severed, the system continues operating smoothly.

10.5 Historical Price Data Service (trueDataHistoricalService.js)

Historical OHLCV candle records are fetched for rendering interactive price charts in `StockChartPopup.jsx`:

- Endpoint: `GET /api/market/prices/:symbol/historical?range=1M`
- Supported Ranges: `1M` (1 Month, 15-min candles), `1Y` (1 Year, daily candles), `5Y` (5 Years, weekly candles), `MAX`.
- Token Caching: Historical REST authentication tokens are acquired from `https://auth.truedata.in/token` and cached in memory with an expiration timestamp to prevent unnecessary re-authentication overhead.

Chapter 11 — Tournament & Paper Trading Engine

11.1 Competitive Paper Trading Architecture

The Tournament engine provides an isolated, competitive trading environment where multiple traders participate in timed trading contests using virtual capital. This enables risk-free paper trading while introducing realistic market conditions, competition pressure, and objective performance metrics.

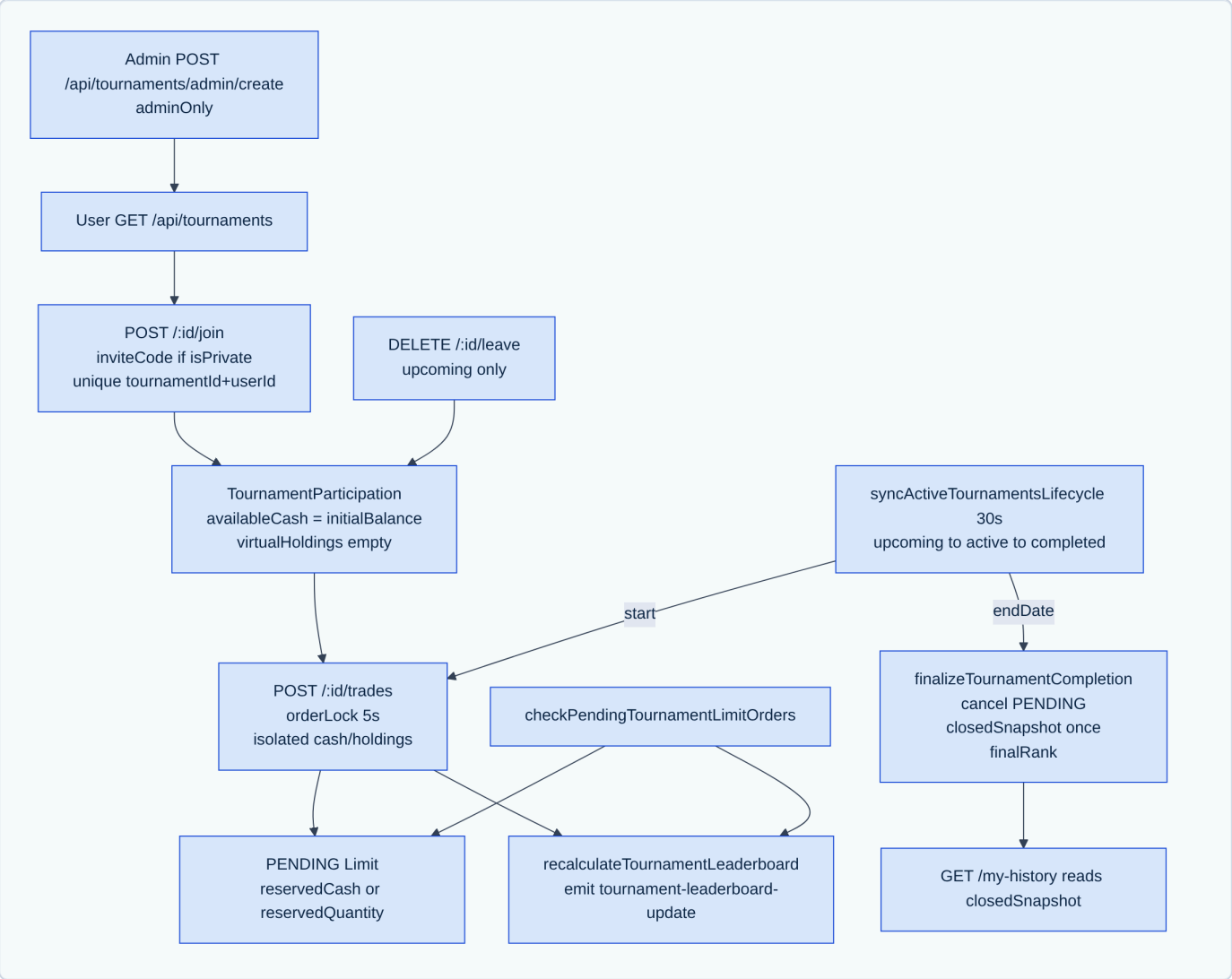


Figure 11.1: Complete tournament lifecycle from creation, joining, virtual trading, to completion and prize settlement.

11.2 Virtual Portfolio Isolation Model

A critical design requirement is that **tournament trading must never contaminate a user's real paper-trading capital**:

```
Real Trading Account (FundsModel)
  └── availableBalance: ₹100,000 (Completely unaffected by tournament losses)

Tournament Participation Record (TournamentParticipationModel)
  ├── tournamentId: "tournament_oct_2026"
  ├── userId: "68c123456789abcdef123455"
  ├── initialBalance: ₹500,000 (Independent virtual tournament capital)
  ├── virtualCash: ₹425,000
  ├── virtualHoldings: [{ symbol: "RELIANCE", qty: 25, avgPrice: 2500 }]
  ├── totalPortfolioValue: ₹512,500
  ├── roi: +2.50%
  └── currentRank: 1
```

11.3 Tournament Lifecycle State Machine

Tournaments transition through three strict states managed by an automated background worker (`setInterval(syncActiveTournamentsLifecycle, 30000)` in `backend/index.js`):

Status	Preconditions	Permitted Actions	State Transitions
upcoming	`currentTime < startDate`	Traders can view details and register/join. Virtual capital allocated.	Transitions to `active` when `currentTime >= startDate`.
active	`startDate <= currentTime < endDate`	Participants can submit virtual buy/sell orders. Real-time ticks update portfolio value.	Transitions to `completed` when `currentTime >= endDate`.
completed	`currentTime >= endDate`	Trading disabled. Final leaderboard calculated and winners locked.	Terminal state. Results preserved for historical review.

11.4 Real-Time Leaderboard & ROI Calculation

Leaderboard rankings are based on **Return on Investment (ROI)** rather than absolute rupee value, ensuring fairness across varying tournament structures:

```
Current Equity = Virtual Cash + Sum of all (Virtual Qty * Current LTP)
Net Return     = Current Equity - Initial Virtual Balance
ROI (%)        = ( Net Return / Initial Virtual Balance ) * 100
```

As live market ticks stream into the backend, the tournament service recalculates participants' portfolio values. Updated ranks are broadcast immediately over WebSockets to all clients in the tournament room:

```
io.to(`tournament:${tournamentId}`).emit('leaderboard-update', {
  tournamentId,
  leaderboard: updatedRankings
});
```

Chapter 12 — AI Assistant & Portfolio Context Injection

12.1 Conversational Financial Intelligence Overview

Zerodha Clone features an integrated conversational AI financial assistant built with Google Gemini (using the modern `@google/genai` SDK). Unlike generic chatbot implementations that offer isolated answers, this assistant operates with **real-time portfolio context injection**: it inspects the authenticated user's active holdings, intraday positions, recent order history, and available cash before answering, allowing it to provide personalized portfolio health assessments, risk evaluations, and trade explanations.

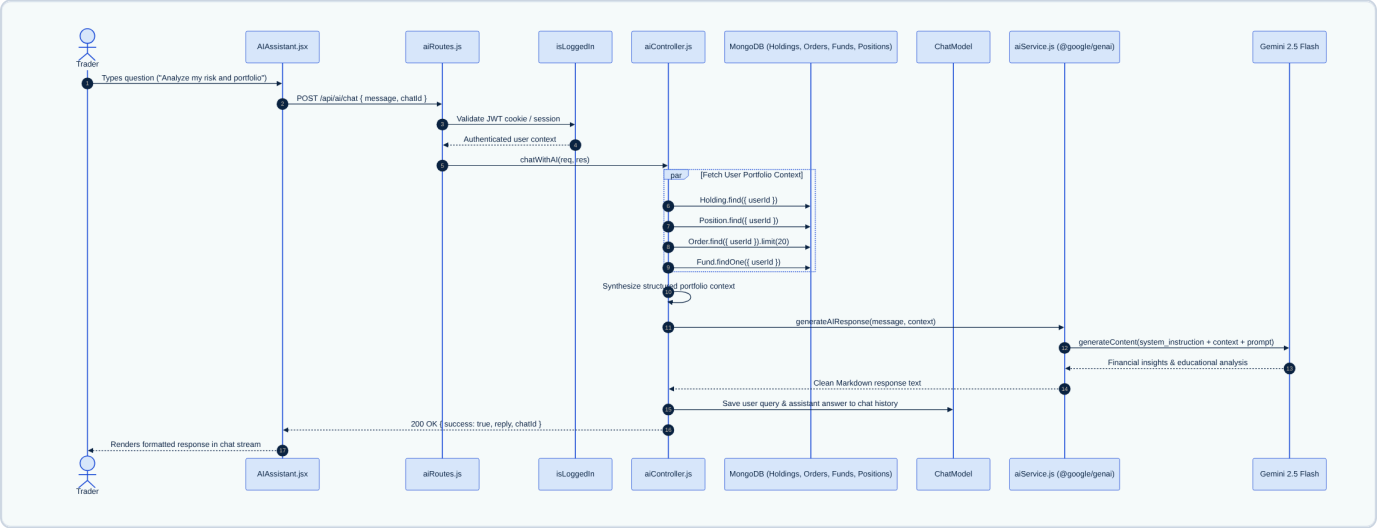


Figure 12.1: Sequence diagram of user query, parallel portfolio context aggregation, Gemini API invocation, and chat persistence.

12.2 Context Aggregation Pipeline (aiController.js)

When a user submits a prompt via `POST /api/ai/chat`, the controller executes a high-speed parallel query to gather the trader's complete financial state:

```
const [holdings, positions, orders, funds, watchlist] = await Promise.all([
  Holding.find({ userId }).lean(),
  Position.find({ userId }).lean(),
  Order.find({ userId }).sort({ createdAt: -1 }).limit(20).lean(),
  Fund.findOne({ userId }).lean(),
  WatchlistList.findOne({ userId, isDefault: true }).lean(),
]);
```

The controller synthesizes these documents into a concise, token-efficient structured context string:

```
--- USER PORTFOLIO CONTEXT ---
Available Capital: ₹75,200 | Used Margin: ₹24,800 | Reserved: ₹0
Holdings:
- RELIANCE: 10 shares @ avg ₹2,450 (Current LTP: ₹2,525, P&L: +₹750 / +3.06%)
- TCS: 5 shares @ avg ₹3,280 (Current LTP: ₹3,350, P&L: +₹350 / +2.13%)
Active Positions: None
Recent Orders:
- BUY 10 RELIANCE @ 2450 (status: completed)
Tracked Watchlist: RELIANCE, TCS, INFY, HDFCBANK, ICICIBANK, SBIN, ITC
------------------------------
```

12.3 System Prompt & Financial Safety Guardrails

The service enforces strict regulatory and ethical boundaries defined in `backend/services/aiService.js`:

```
const SYSTEM_INSTRUCTION = `
You are an AI assistant inside a stock trading and portfolio management application.
Your role is to help users understand:
- Their portfolio, holdings, positions, orders, funds, profit and loss
- Stock market concepts and trading terminology

Rules:
1. Give clear and simple explanations.
2. Use the user's portfolio data when it is provided in the prompt.
3. Never invent portfolio data, prices, orders, funds, or transactions.
4. If required information is not provided, clearly say that the information is unavailable.
5. Never execute, place, modify, or cancel a trading order.
6. Never claim that an order has been executed unless the backend explicitly provides that information.
7. Do not ask for passwords, OTPs, API keys, or other sensitive credentials.
8. For financial topics, provide educational information and avoid presenting predictions as guaranteed
outcomes.
9. Keep responses concise and useful.
`;
```

12.4 Chat History Persistence & Session Management

All conversations are stored in MongoDB via `ChatModel.js`:

- Each conversation document maintains an array of message objects: `{ role: 'user' | 'assistant', content: String, timestamp: Date }`.
- If the user passes an existing `chatId`, new messages are appended to that document, maintaining multi-turn context.
- If no `chatId` is supplied, a new chat document is created with a generated title (e.g., derived from the user's initial question).
- Users can review past conversation threads (`GET /api/ai/chats`) or delete obsolete history (`DELETE /api/ai/chats/:chatId`).

12.5 Resilient Error Handling & API Key Fallback

If `GEMINI_API_KEY` is absent in development or if external rate limits occur, `aiService.js` captures the condition gracefully without returning 500 errors to the client, providing clear educational guidance on how to enable the generative AI key.

Chapter 13 — Advanced Ecosystem Features

13.1 Multi-Watchlist System (watchlistsController.js)

Similar to Zerodha Kite's multi-watchlist tab navigation (Watchlist 1 through 7), the platform allows traders to organize equities into distinct custom lists:

- Up to **7 watchlists per user**, each holding up to **50 stock symbols**.
- Custom watchlist naming (e.g. "Banking", "IT Sector", "High Beta").
- Symbol re-ordering via drag-and-drop or sequential array storage.
- Fast hovering quick-action buttons for instantaneous Buy/Sell, market depth, and chart expansion.

13.2 Price Target Alerts (priceAlertController.js)

Traders can set automated alerts for any stock based on condition triggers:

- **Conditions:** `GREATER_THAN_OR_EQUAL` (Price >= Target) or `LESS_THAN_OR_EQUAL` (Price <= Target).
- **Evaluation Engine (`evaluateAlerts`):** Whenever TrueData emits a live price tick, `evaluateAlerts(symbol, ltp)` inspects active alerts for that symbol.
- **Notification Trigger:** When a condition triggers, the alert is marked `status: 'triggered'`, a high-priority in-app notification is recorded, and an event is dispatched over Socket.IO to the user's browser.

13.3 In-App Notifications & Preference Control

The notification subsystem (`NotificationModel.js`) delivers real-time and persistent alerts categorized by priority (`low`, `normal`, `high`):

- **Event Categories:** Order filled, limit order triggered, price alert hit, fund deposit/withdrawal confirmed, tournament status change.
- **Granular User Preferences (`NotificationPreferenceModel.js`):** Users can toggle individual alert types on or off (e.g., disable deposit alerts while keeping limit order fill alerts enabled).
- **Unread Tracking:** Distinct read/unread timestamps and a bulk "Mark All as Read" endpoint (`POST /api/notifications/read-all`).

13.4 Portfolio Analytics & Performance Metrics (portfolioAnalyticsService.js)

Provides data-driven feedback on user trading performance over time:

- **Asset Allocation:** Sector and asset distribution breakdown (Large Cap, Mid Cap, Cash Reserve) visualized via Recharts pie charts.
- **Profit Factor:** `Total Gross Profits / Total Gross Losses`.
- **Win-Loss Ratio:** `Profitable Closed Trades / Total Closed Trades * 100`.
- **Historical Equity Curve:** Daily portfolio equity snapshots captured via `PortfolioSnapshotModel.js` to chart historical account growth.

13.5 Risk Guard: Quantitative Risk Management (riskService.js)

To prevent catastrophic emotional drawdowns (tilt trading), the platform implements an active **Risk Guard** system:

- **Daily Loss Limit:** Maximum loss permitted in a single trading day (e.g. ₹5,000). If hit, order placement triggers a warning or temporary restriction.
- **Maximum Position Size Cap:** Restricts the maximum capital allowed in any single trade (e.g. max 20% of total portfolio).

- **Leverage Caps:** Configurable maximum leverage allowed for intraday MIS positions.

13.6 Trading Journal (journalService.js)

Professional trading requires reflective review of setups, thesis, and emotional discipline:

- **Rich Notes & Setup Tags:** Traders record technical rationale (Breakout, Pullback, Mean Reversion, Gap Fill).
- **Emotional Tags:** Traders log emotional states (FOMO, Revenge Trade, Patient, Confident, Hesitant) to identify psychological patterns correlating with losses.
- **Trade Linking:** Journal entries can be directly linked to executed order IDs to pair qualitative analysis with objective execution data.

13.7 Strategy Backtesting Engine (backtestEngine.js)

A quantitative historical simulation engine allowing traders to validate technical rules before risking capital:

- **SMA Crossover Strategy (`smaCrossover.js`):** Simulates Fast Simple Moving Average (e.g. 9-period SMA) crossing Slow Simple Moving Average (e.g. 21-period SMA).
- **Signal Generation:** Fast SMA crosses above Slow SMA -> BUY Signal. Fast SMA crosses below Slow SMA -> SELL Signal.
- **Simulation Performance Report:** Generates Net Return %, Maximum Drawdown %, Win Rate %, Total Trades Executed, and Sharpe Ratio stored in `BacktestHistoryModel.js`.

Chapter 14 — Admin Operations, Concurrency & Reliability

14.1 Role-Based Access Control (RBAC) & Admin Middleware

Administrative capabilities are strictly guarded using role-based access control. The `UserModel` defines a `role` enum (`'trader' | 'admin'`). Sensitive endpoints, such as tournament creation, manual square-offs, and platform settings, are protected by `adminMiddleware.js`:

```
export const adminOnly = (req, res, next) => {
  if (!req.user || req.user.role !== "admin") {
    return res.status(403).json({
      success: false,
      message: "Forbidden: Administrative privileges required."
    });
  }
  next();
};
```

The system includes administrative utility scripts (`backend/scripts/setAdminRole.js`) allowing system operators to safely grant administrative permissions by email address.

14.2 Tournament Administration Interface (TournamentAdmin.jsx)

Administrators access a dedicated management portal (`/admin/tournaments`) providing:

- **Tournament Creation Form:** Configuring title, description, entry fee, initial virtual balance (e.g. ₹5,00,000), registration window, competition start date/time, and end date/time.
- **Competition Lifecycle Controls:** Ability to view enrolled participants, inspect aggregate virtual portfolios, and manually trigger emergency freezes or lifecycle status transitions.
- **Audit Trails (`adminAudit.js`):** Every administrative mutation logs the executing administrator's ID, action timestamp, IP address, and changed parameters for compliance and debugging.

14.3 Concurrency Control & Race Condition Mitigation

High-frequency financial applications face severe concurrency vulnerabilities if state mutations are not carefully sequenced. Zerodha Clone implements multiple defense-in-depth mechanisms:

1. Anti-Double-Spend with Two-Phase Limit Order Locking

> **Vulnerability:** If two TrueData market ticks arrive within 2 milliseconds of each other while an order is pending, two concurrent worker threads might both trigger execution for the same order, purchasing double the intended shares and causing balance corruption.

Mitigation: `checkPendingLimitOrders.js` uses an atomic atomic state transition from `'pending'` to `'processing'`:

```
const lockedOrder = await Order.findOneAndUpdate(
  { _id: order._id, status: 'pending' },
  { $set: { status: 'processing' } },
  { new: true }
);
if (!lockedOrder) return; // Second thread skipped immediately!
```

2. Exactly-Once Reservation Release Guard

Every order includes a boolean flag: `reservationReleased: Boolean` (default: `false`). When cash or stock is returned or converted to used margin:

```
const order = await Order.findOneAndUpdate(
  { _id: orderId, reservationReleased: false },
  { $set: { reservationReleased: true, status: 'cancelled' } },
  { session }
);
if (!order) throw new Error("Reservation already released or order settled.");
```

This guarantees that user refunds can never be processed twice, even if multiple cancellation clicks occur in rapid succession.

3. Multi-Document ACID Transactions

All operations updating both ledger balances and asset inventories run inside native MongoDB replica-set transactions. If an order insertion fails after balance debiting, the entire transaction rolls back cleanly, ensuring zero financial discrepancy.

14.4 Error Handling Architecture

- **Centralized Error Class (`AppError.js`):** Differentiates operational errors (known client errors like invalid inputs or insufficient margin) from fatal programmer bugs.
- **Fail-Fast DB Resilience:** Mongoose is configured with `bufferCommands: false`. If the database connection drops, operations fail fast with meaningful error responses rather than hanging for 30-second timeouts.
- **Graceful Degradation:** If external APIs (Twilio SMS or TrueData feeds) encounter downtime, fallbacks allow core application navigation and mock testing to proceed without crashes.

14.5 Known Limitations & Future Architecture Roadmap

Current Limitation	Technical Cause	Recommended Production Solution
Single-Instance Socket.IO	Socket rooms are currently in-memory on the Node server.	Implement `@socket.io/redis-adapter` for horizontal cluster scaling.
In-Memory Polling Worker	Tournament and limit order checks run via `setInterval`.	Migrate to distributed message queues (BullMQ / Redis) for background jobs.
Relational Ledger Querying	Double-entry accounting relies on document array transactions.	Adopt PostgreSQL (AI Studio Cloud SQL) for strict transactional ledgering.

Chapter 15 — Technical Interview Preparation Masterclass

This chapter provides **100 project-grounded interview questions with clear, comprehensive answers**. Each question tests architectural judgment, deep implementation knowledge, concurrency reasoning, and systems design derived directly from this Zerodha Clone codebase.

Part I: Architecture & System Design (Questions 1–15)

1. What is the high-level architecture of this Zerodha Clone project?

It is a full-stack, event-driven trading platform consisting of a React 19 Single Page Application (SPA) frontend, an Express 5 (Node.js 22) REST API gateway, Socket.IO real-time event streaming, and a MongoDB persistence tier. It integrates with TrueData for live/historical NSE market feeds, Twilio Verify for mobile OTP, and Google Gemini for portfolio-aware financial AI.

2. Why did you choose a Controller-Service-Repository pattern rather than writing queries directly inside controllers?

Controllers should only handle HTTP transport concerns (parsing status codes, reading cookies, returning JSON). Placing business rules (such as limit order execution, balance calculations, and multi-document transactions) in dedicated services enables unit testing, code reuse across HTTP routes and background workers, and prevents tight coupling to the web framework.

3. How does the application handle both live and development/offline environments?

It uses a defensive fallback architecture: if MongoDB, TrueData, or Twilio are unconfigured or temporarily offline, the server boots in resilient mode with mock baseline prices, default ledger balances, and local OTP codes (123456), ensuring zero crashes during local inspection or testing.

4. How do you prevent Cross-Origin Resource Sharing (CORS) issues in production?

The Express backend mounts Vite middlewares in development and serves pre-built static client files in production on the exact same port (3000). Because both the API and client share the same domain and port origin, cross-origin security blocks are eliminated entirely.

5. Why is a monolithic wallet balance inadequate for a trading application?

A single balance cannot represent capital locked in pending limit orders or funds tied up in active equity cost bases. We implement a three-tier model: availableBalance (liquid free cash), usedBalance (deployed in active holdings), and reservedBalance (locked for pending limit orders).

6. How does the system handle real-time market data distribution?

TrueData pushes touchline ticks over WebSockets into the backend. The backend updates an in-memory price map, evaluates limit order trigger conditions, and dispatches real-time events over Socket.IO to connected browser clients.

7. How are user notifications routed to specific individuals without broadcasting to everyone?

When a client connects to Socket.IO, it emits `join-user-room` with its authenticated user ID. The server places that socket connection into room `user:{userId}`. Notifications are emitted strictly via `io.to('user:' + userId).emit('new-notification', data)`.

8. How do you isolate tournament paper trading from real paper-trading portfolios?

Tournament records are maintained in completely separate collections (`TournamentParticipationModel` and `TournamentOrderModel`). A user's main `Funds` balance is never affected by tournament entry positions or losses.

9. What happens if the backend server crashes while a limit order is executing?

Because all order mutations and balance debit/credits run inside atomic MongoDB sessions (`session.startTransaction()`), an unexpected server crash automatically triggers transaction rollback by MongoDB, leaving no orphan balances.

10. How would you scale the Socket.IO tier across multiple server instances?

By introducing the `@socket.io/redis-adapter`. This connects multiple Node.js instances to a shared Redis Pub/Sub backplane so that room broadcasts emitted from Server A reach sockets connected to Server B.

11. Why is HTTP polling avoided for order status updates?

Polling introduces severe server load, redundant database queries, and latency. Socket.IO event push guarantees instant, sub-50ms UI updates upon order fill with minimal network overhead.

12. How does the system enforce separation between CNC delivery and MIS intraday products?

CNC requires 100% upfront cash, prohibits short-selling, and saves directly into `holdings`. MIS allows leverage, supports short-selling, and records into `positions` with automated square-off requirements.

13. What is the role of `server.js` versus `backend/index.js`?

`backend/index.js` encapsulates the Express route configurations, database initialization, and socket setup. `server.js` serves as the root application entry point that mounts frontend Vite middlewares alongside the backend.

14. What design considerations were made for mobile and responsive layouts?

The dashboard features a collapsible sidebar that transitions to an icon dock below 991px, grid layouts that reflow from 4 cards to 1 card on mobile, and responsive touchline action buttons.

15. How are database migrations and initial seed data handled?

`backend/data/init.js` and `backend/data/data.js` contain programmatic seed utilities for holdings, positions, funds, and orders, enabling instantaneous environment hydration.

Part II: React & Frontend Architecture (Questions 16–30)

16. Why was React 19 selected for the frontend?

React 19 offers improved asynchronous state handling, enhanced batching, improved hydration performance, and seamless compatibility with modern Vite compilation pipelines.

17. Explain how `AppContext.jsx` acts as the single source of truth.

It consolidates user data, holdings, open positions, ledger balances, market prices, and notifications into a single React context provider, exposing state and updater functions to all child components without prop drilling.

18. What are `orderVersion`, `holdingVersion`, and `fundVersion` state counters in AppContext?

They are reactive dependency counters. When an order completes or funds are added, incrementing `orderVersion` triggers `useEffect` hooks across disparate components (Orders page, Holdings page, Header balance badge) to re-fetch fresh state in parallel.

19. How does `ProtectedRoute.jsx` enforce client-side navigation security?

It queries `GET /api/auth/me` with `{ withCredentials: true }` upon mounting. If the session is invalid, it redirects the browser to `/login`. If authenticated, it renders `children` (the dashboard layout).

20. How does `TradePopup.jsx` compute dynamic margin requirements?

It binds a listener to quantity and price inputs: `Required Margin = Quantity * (OrderType === 'MARKET' ? LTP : LimitPrice)`. It compares this against `availableBalance` in real time, disabling the submit button if funds are insufficient.

21. How is dark mode implemented across the frontend?

The user's theme selection is stored in `localStorage` and applied to the root `<html>` tag as `data-theme="dark"`. CSS custom variables defined in `App.css` automatically adapt card backgrounds, borders, and text colors.

22. How are charts rendered in the portfolio analytics dashboard?

Using Recharts components (`ResponsiveContainer`, `PieChart`, `AreaChart`, `BarChart`) bound to computed asset allocation and P&L timeline arrays.

23. How does the frontend prevent UI lag when receiving high-frequency price updates?

Market tick updates modify state inside a keyed object/Map (`marketPrices[symbol]`), ensuring React only re-renders components specifically subscribing to the changed symbol rather than the entire page.

24. What is the purpose of `FundToast` and `OrderToast`?

They provide non-blocking floating status feedback confirming order placement, limit order triggers, and wallet balance updates without obscuring user interactions.

25. How does `MarketWatchlist.jsx` support multi-watchlist switching?

It renders numeric tab pills (1 to 7). Clicking a pill sets `activeWatchlistId`, which filters displayed stocks and queries the backend for custom symbol orderings.

26. How are modals managed across the dashboard?

Modal open/close states (TradePopup, StockChartPopup, PriceAlertModal, StockDetailsPopup) are managed in AppContext, allowing any button across watchlists or positions to launch a modal.

27. How does the client handle network disconnection with Socket.IO?

Socket.IO Client is initialized with `reconnection: true`, exponential backoff (`reconnectionDelay: 1000`, `reconnectionDelayMax: 5000`), and automatically re-joins user rooms upon reconnecting.

28. Why use `axios` over native `fetch` in this project?

Axios provides automated JSON body serialization/parsing, unified interceptors for credentials, and consistent error response normalization via `error.response.data`.

29. How does `AIAssistant.jsx` stream and format generative responses?

It uses `react-markdown` to parse Gemini's markdown output, rendering syntax-highlighted code blocks, financial bullet points, and clean typography.

30. What strategy is used to prevent memory leaks in React `useEffect` hooks?

All timers (such as OTP countdown clocks) and Socket.IO event listeners return explicit cleanup functions (e.g. `socket.off('new-notification')` and `clearInterval(timer)`).

Part III: Node.js, Express & Middleware (Questions 31–45)

31. What are the key architectural improvements in Express 5 used in this backend?

Express 5 natively handles rejected promises and asynchronous errors in route handlers without requiring manual `try/catch` and `next(err)` boilerplates, simplifying async controller code.

32. How does `isLoggedInMiddleware.js` extract and authenticate the user?

It extracts `req.cookies.token`, verifies the signature using `jwt.verify(token, JWT_SECRET)`, queries `User.findById(decoded.userId)`, confirms `tokenVersion` matches, and attaches `req.user`.

33. Why is Joi used for payload validation instead of manual if-else statements?

Joi provides declarative, schema-based validation (`orderValidationSchema.js` and `userValidationSchema.js`), validating types, minimums, maximums, and allowed string enums before controllers execute.

34. How does `adminMiddleware.js` protect administrative routes?

It executes immediately following `isLoggedIn` and asserts `req.user.role === 'admin'`. If false, it terminates the request with HTTP 403 Forbidden.

35. What is the role of `cookie-parser` in this architecture?

It parses the HTTP `Cookie` header string into a structured JavaScript object on `req.cookies`, allowing clean retrieval of the signed authentication cookie.

36. How does the server handle uncaught exceptions and database disconnects?

The global error middleware inspects `err.name === 'MongooseError'`. It logs the incident and returns a graceful 503 or empty array fallback rather than crashing the process.

37. How does the Node process schedule background tournament synchronization?

In `backend/index.js`, an interval worker runs every 30 seconds (`setInterval(syncActiveTournamentsLifecycle, 30000)`), executing automated tournament state transitions.

38. Explain how Client IDs are generated in `generateClientId.js`.

It generates a 2-character prefix ('ZR') followed by 4 random alphanumeric characters, checking the database in a loop to ensure uniqueness before assigning it to a new trader.

39. Why are passwords excluded from default User queries?

The `password` schema definition specifies `select: false`. This prevents hashed passwords from leaking in API responses or user profile queries unless explicitly requested via `.select('+password')`.

40. How is CORS configured to allow cookies across origins in local testing?

`cors({ origin: true, credentials: true })` reflects the request origin and sets `Access-Control-Allow-Credentials: true`, permitting browser cookie transmissions.

41. What is the difference between `process.exitCode = 1` and `process.exit(1)`?

Setting `process.exitCode = 1` allows the Node.js event loop to complete pending I/O operations and close active database sessions gracefully before exiting, avoiding abrupt socket terminations.

42. How does the backend prevent brute-force OTP attempts?

`OtpChallengeModel` enforces a limit of 3 challenge requests per 5-minute window per phone number, returning HTTP 429 Rate Limit Exceeded if violated.

43. What is the function of `adminAudit.js`?

It records administrative mutations (such as tournament status modifications) to a persistent audit log, logging admin ID, timestamp, IP, and payload for compliance.

44. How does the application leverage ES Modules (`import/export`) natively?

By setting `"type": "module"` in `package.json`, allowing modern Node.js top-level await and clean ES6 module imports without Babel transpilation.

45. How does the server handle graceful shutdown?

By listening to `SIGTERM` and `SIGINT`, stopping new HTTP requests, finishing in-flight transactions, closing database connections, and exiting cleanly.

Part IV: MongoDB, Mongoose & Transactions (Questions 46–60)

46. Why is MongoDB suitable for this trading application?

Its dynamic document model easily represents complex financial objects (such as orders with execution histories and tournament portfolios with nested holdings) while supporting native multi-document ACID transactions.

47. How are multi-document transactions initiated in Mongoose?

Via `const session = await mongoose.startSession(); session.startTransaction();`. All queries pass `{ session }`, and operations commit via `await session.commitTransaction()`.

48. Why is `bufferCommands: false` configured on Mongoose?

By default, Mongoose buffers queries indefinitely if the database disconnects. Setting `bufferCommands: false` causes operations to fail fast immediately, allowing fallback code to respond promptly.

49. What indexes are placed on the `Orders` collection?

Indexes on `userId`, `symbol`, and a compound index on `{ symbol: 1, status: 1 }` to enable fast background matching of pending limit orders on live price ticks.

50. How does the `Holdings` model handle partial sell operations?

It executes `$inc: { quantity: -sellQty }`. If `quantity` drops to 0, the holding document is removed via `Holding.deleteOne({ _id })`.

51. What is the significance of `.lean()` in Mongoose queries?

It instructs Mongoose to return plain, lightweight JavaScript objects rather than heavy Mongoose Document instances with change tracking, reducing memory consumption by up to 70%.

52. How does `UserModel` guarantee uniqueness on mobile and email?

By declaring `unique: true` on both fields, backed by MongoDB unique B-tree indexes that prevent concurrent duplicate registrations.

53. How is the double-entry accounting ledger maintained in `FundTransactionModel`?

Every balance change (deposit, withdrawal, order fulfillment) writes an immutable transaction document recording `userId`, `amount`, `type`, `status`, and `createdAt`.

54. What prevents `availableBalance` from going below zero in concurrent requests?

Atomic query filters: `Funds.findOneAndUpdate({ userId, availableBalance: { $gte: tradeAmount } }, { $inc: { availableBalance: -tradeAmount } })`. If funds are insufficient, no document matches.

55. What is the purpose of `PortfolioSnapshotModel.js`?

It stores end-of-day portfolio valuation snapshots (invested, current, cash, total equity) for generating historical performance curves and drawdown charts in the analytics dashboard.

56. How does the `ChatModel` structure message history?

It stores messages as an embedded subdocument array `[{ role: 'user' | 'assistant', content: String, timestamp: Date }]` indexed by `userId` and `chatId`.

57. Explain the schema design of `WatchlistListModel`.

Each document represents a named watchlist containing `userId`, `name`, `isDefault: Boolean`, and an array of symbol subdocuments `[{ symbol: String, companyName: String }]`.

58. How does `OtpChallengeModel` auto-expire stale challenges?

By utilizing a MongoDB TTL (Time-To-Live) index on `expiresAt` that automatically purges expired challenge documents from the database.

59. What happens if a MongoDB transaction fails halfway through?

The catch block invokes `await session.abortTransaction()`, rolling back all staged writes and restoring the database to its pre-transaction state.

60. How are relations modeled between Users, Funds, Holdings, and Orders?

Via referenced ObjectId linking (`userId`). Funds has a 1-to-1 relationship with User, while Orders, Holdings, and Positions have 1-to-Many relationships with User.

Part V: Trading Mechanics, Order Execution & Math (Questions 61–75)

61. What is the difference between a Market Order and a Limit Order?

A Market Order executes immediately at the current market LTP. A Limit Order executes only when market price reaches or improves upon a specified limit trigger price.

62. Walk through the cash reservation lifecycle of a Limit Buy order.

1) Placement: `availableBalance -= qty * limitPrice`, `reservedBalance += qty * limitPrice`. 2) Trigger: Executed at LTP. `reservedBalance -= reservedAmount`, `usedBalance += qty * LTP`. 3) Savings Refund: Leftover `(limitPrice - LTP) * qty` returns to `availableBalance`.

63. How is stock reserved on a Limit Sell order?

The shares are transferred from unencumbered `quantity` to `reservedQuantity` inside `Holding`. This prevents the trader from selling those same shares in a separate market order while the limit order is active.

64. What is the mathematical formula for Volume-Weighted Average Purchase Price?

`((Existing_Qty * Existing_Avg_Price) + (New_Qty * New_Price)) / (Existing_Qty + New_Qty)`.

65. If a user owns 20 shares of Reliance at ₹2,400 and buys 10 more at ₹2,700, what is the new average price?

Total Invested = (20 * 2400) + (10 * 2700) = 48,000 + 27,000 = ₹75,000. Total Shares = 30. New Average Price = 75,000 / 30 = ₹2,500 per share.

66. How does the system prevent double execution of a limit order when multiple ticks arrive?

Using two-phase atomic locking: the background worker queries for `status: 'pending'` and immediately transitions it to `'processing'`. A concurrent tick finding no order with status `'pending'` simply skips execution.

67. What is `reservationReleased` and why is it essential?

It is a boolean latch on the Order model. It ensures that unlocking reserved cash or shares occurs strictly once, preventing double refunds on duplicate cancellation clicks.

68. How is Day P&L calculated for an individual stock?

`Day P&L = Quantity * (Current Market Price - Previous Day Close Price)`.

69. How is Total Unrealized P&L calculated for a holding?

`Total P&L = Quantity * (Current Market Price - Average Purchase Price)`. Percentage return is `(Total P&L / (Quantity * Average Purchase Price)) * 100`.

70. What happens when a user cancels a pending Limit Buy order?

`cancelLimitOrderService.js` confirms status is `pending`, marks `reservationReleased: true`, decrements `reservedBalance` by `reservedAmount`, and increments `availableBalance` by `reservedAmount`.

71. Can a user withdraw funds that are currently in `reservedBalance`?

No. The withdrawal endpoint strictly checks `withdrawalAmount <= availableBalance`. Reserved funds are legally encumbered by pending market commitments.

72. How is intraday leverage modeled for MIS orders?

For MIS trades, required margin is computed as `(qty * price) / leverageFactor` (e.g. 5x leverage requires only 20% upfront cash), allowing intraday day-trading simulation.

73. What is short selling in MIS and how does the engine handle it?

A trader sells shares they do not own, creating a Position with `type: 'sell'`. They profit if price drops and must buy back (square-off) the shares before market close.

74. How does the engine handle orders placed when market prices fluctuate wildly (slippage)?

Market orders execute at the latest cached LTP. In a live production exchange, orders would route to an exchange matching engine with price tolerance bands.

75. What validation rules are applied to order placement payloads?

`orderValidation.js` enforces: `symbol` must be valid string, `qty` must be positive integer >= 1, `price` must be positive number >= 0.05, `mode` must be 'BUY' or 'SELL', `orderType` must be 'MARKET' or 'LIMIT'.

Part VI: Authentication, Tokens & Security (Questions 76–85)

76. Why store JWTs in HTTP-only cookies instead of `localStorage`?

`localStorage` is accessible to any JavaScript running on the page, making it vulnerable to token theft via Cross-Site Scripting (XSS). HTTP-only cookies cannot be read by JavaScript.

77. How does the `sameSite: 'lax'` attribute protect against CSRF attacks?

It prevents the browser from sending authentication cookies on cross-site subrequests (such as malicious third-party POST forms or embedded images), mitigating Cross-Site Request Forgery.

78. How does `tokenVersion` enable logging out all devices?

When the user selects "Logout All Sessions", the server increments `user.tokenVersion` in MongoDB. Existing tokens on other devices carry the old version number and fail validation on their next request.

79. Why is `bcryptjs` used with a work factor of 10?

Bcrypt is a slow hashing algorithm designed to resist brute-force hardware attacks. Work factor 10 requires ~100ms per hash, striking an optimal balance between security and server throughput.

80. Explain the role of the `verificationToken` in mobile registration.

It is a short-lived signed JWT issued upon successful SMS OTP verification. Including it in the final registration payload proves cryptographically that the mobile number was verified before creating the user account.

81. How does the application prevent account enumeration during login?

When OTP dispatch or password verification fails, the API returns a generic error message ("Invalid mobile number or credentials") rather than disclosing whether the account exists.

82. What claims are encoded in the primary authentication JWT?

`{ userId: user._id, role: user.role, tokenVersion: user.tokenVersion }` signed with `JWT_SECRET` and a 24-hour expiration (`expiresIn: '1d'`).

83. How are sensitive configuration secrets protected in source control?

All secrets (`JWT_SECRET`, `MONGO_URI`, `GEMINI_API_KEY`, `TWILIO_AUTH_TOKEN`) are loaded from `.env` via `dotenv` and explicitly excluded from git tracking via `.gitignore`.

84. How does the system guard against NoSQL injection attacks?

Mongoose schema type casting and Joi input validation ensure query parameters match expected primitive types, preventing attackers from injecting operator objects (e.g. `{"$gt": ""}`).

85. What is the role of `passport.js` in this architecture?

It encapsulates the Passport Local Strategy for email/password authentication, verifying user credentials and serializing user identity.

Part VII: AI Assistant & External Integrations (Questions 86–95)

86. Which Google GenAI SDK is utilized in this project?

The modern `@google/genai` TypeScript/JavaScript SDK, instantiating `new GoogleGenAI({ apiKey })` and targeting the `gemini-2.5-flash` model.

87. How is real-time portfolio context provided to the Gemini model?

Before calling the AI model, `aiController.js` queries user holdings, active positions, recent orders, and balances in parallel, formatting them into an informative system prompt string.

88. What guardrails prevent the AI assistant from executing unauthorized trades?

System instructions explicitly command the model: "Never execute, place, modify, or cancel a trading order." The AI assistant backend does not have access to order mutation function bindings.

89. How are AI conversations persisted in MongoDB?

Each conversation is stored in `ChatModel.js` with an array of user and assistant messages, enabling users to switch between conversation threads or review past financial analyses.

90. How does the AI service handle missing API keys or external service limits?

It catches API errors gracefully in `aiService.js`, returning informative financial guidance on technical indicators rather than throwing unhandled 500 exceptions to the UI.

91. How does the TrueData historical service optimize token usage?

It requests an OAuth token from TrueData Auth REST API and caches it in memory with an expiration timestamp (`historicalTokenExpiresAt`), reusing it across subsequent historical candle queries.

92. What ranges are supported for historical stock charts?

`1M` (1 month, intraday 15-min candles), `1Y` (1 year daily candles), `5Y` (5 year weekly candles), and `MAX`.

93. How are Indian stock symbols normalized across TrueData and user inputs?

`normalizeSymbol` trims whitespace, converts to uppercase, and strips `NSE:` or `BSE:` prefixes, guaranteeing consistent cache keys and query matching.

94. How does the tournament engine rank competitors?

By Return on Investment: `ROI = ((Current Portfolio Value - Initial Balance) / Initial Balance) * 100`. Traders are sorted descending by ROI.

95. How does the SMA Crossover backtesting strategy generate signals?

It calculates a Fast SMA (9-period) and Slow SMA (21-period) on historical closing prices. A crossover of Fast above Slow generates a BUY; Fast below Slow generates a SELL.

Part VIII: Production Readiness & Debugging (Questions 96–100)

96. How do you debug an issue where a limit order failed to execute?

1) Verify order status in MongoDB (`pending` vs `processing`). 2) Inspect TrueData price cache for the symbol's LTP. 3) Check if LTP crossed the limit price. 4) Review logs in `checkPendingLimitOrders.js` for lock acquisition or session rollback errors.

97. How would you diagnose an unexpected balance discrepancy?

Audit the `FundTransactionModel` collection for the user. Sum all deposits minus withdrawals and compare against `availableBalance + usedBalance + reservedBalance`.

98. How do you prevent memory leaks in Node.js when processing real-time WebSocket ticks?

By storing ticks in a fixed-size `Map` keyed by symbol (updating in-place) rather than appending to an unbounded array, keeping memory usage constant regardless of market tick volume.

99. How is the application built and bundled for production deployment?

`npm run build` runs `vite build`, bundling React components into optimized, minified CSS and JavaScript chunks inside `dist/`. The Node server serves these static assets directly.

100. If an interviewer asks: 'What is the proudest engineering accomplishment in this project?', how would you answer?

'Designing and implementing the institutional three-tier balance ledger and two-phase limit order reservation engine. Solving real-world concurrency issues—such as preventing double-spending on simultaneous market ticks, executing atomic refunds on limit fills, and isolating tournament capital—demonstrated deep full-stack systems engineering beyond standard CRUD applications.'

Chapter 16 — Practical Exercises, Glossary & Revision Cheat Sheet

16.1 Hands-On Coding Exercises

These exercises reinforce your understanding of core financial engineering and full-stack patterns implemented in this codebase:

Exercise 1: Implement a Stop-Loss Market (SL-M) Order Type

Goal: Add support for Stop-Loss Market (SL-M) orders that remain pending until a trigger price is hit, at which point they execute immediately as market orders.

- **Hint:** Add `'SL-M'` to `orderType` enum in `OrdersModel.js` and add a `triggerPrice` field.
- **Hint:** In `createOrderServices.js`, reserve funds or shares based on trigger price. In `checkPendingLimitOrders.js`, evaluate: For Buy SL-M, trigger when `LTP >= triggerPrice`; For Sell SL-M, trigger when `LTP <= triggerPrice`.
- **Learning Outcome:** Master conditional order triggers and multi-state order lifecycles.

Exercise 2: Implement Real-Time Margin Call Notification

Goal: If a user's total unrealized intraday loss in MIS positions exceeds 80% of their available margin, dispatch an urgent high-priority notification via Socket.IO.

- **Hint:** Create a helper in `riskService.js` that sums `(CurrentPrice - AvgPrice) * Qty` across open positions whenever TrueData ticks arrive.
- **Hint:** If `unrealizedLoss > 0.8 * availableBalance`, emit `margin-warning` over Socket.IO to room `user:{userId}`.
- **Learning Outcome:** Understand risk management thresholds and proactive event-driven alerts.

Exercise 3: Add Trailing Stop Loss to Strategy Backtesting

Goal: Enhance `backtestEngine.js` to support dynamic trailing stop-loss percentages (e.g. 2% below highest peak price achieved since position entry).

- **Hint:** Maintain a `highestHigh` variable updated on each candle while in an active position. Calculate `stopPrice = highestHigh * (1 - trailPercent)`. Exit if candle low crosses below `stopPrice`.
- **Learning Outcome:** Experience dynamic risk-reward modeling in quantitative trading simulations.

16.2 Comprehensive Financial & Tech Glossary

Term	Definition & Project Context
CNC (Cash 'N Carry)	Equity delivery product type where shares are purchased with 100% upfront capital and held indefinitely in the user's holdings portfolio.
MIS (Margin Intraday Square-off)	Intraday leveraged trading product where positions must be closed before market session close. Supported in the `positions` collection.
LTP (Last Traded Price)	The latest price at which a stock transaction occurred on the exchange. Broadcast by TrueData WebSocket ticks.
Limit Order	An order to buy or sell a stock at a specified price or better. Requires capital or stock reservation in the ledger while pending.
Touchline Tick	Live top-of-book market feed data containing symbol, LTP, volume, open, high, low, and previous close.

Term	Definition & Project Context
VWAP / Average Price	Volume-Weighted Average Price formula used to compute cost basis across multiple purchases of the same stock.
Three-Tier Ledger	The financial model separating liquid capital (`availableBalance`), deployed capital (`usedBalance`), and locked commitments (`reservedBalance`).
Token Versioning	A database-backed session invalidation pattern enabling immediate revocation of all active JWTs when logging out of other devices.
Socket.IO Room	An isolated channel for targeted message broadcasts (e.g. `user:{userId}` for personal notifications and `tournament:{id}` for leaderboards).
Context Injection	The technique of querying a user's holdings, positions, and orders on-the-fly and injecting them into the Gemini AI system prompt for personalized responses.

16.3 Quick Revision Cheat Sheet

```
┌──────────────────────────┬──────────────────────────────────────────────┐
| Area                     | Quick Summary & Key Files                    |
├──────────────────────────┼──────────────────────────────────────────────┤
| Architecture             | React 19 + Express 5 + MongoDB + Socket.IO on port 3000|
| Authentication           | Mobile OTP (Twilio/Mock) -> JWT in HTTP-Only Cookie    |
| Order Flow               | UI -> orderController -> createOrderServices -> Mongo  |
| Limit Orders             | Lock cash/shares -> Live tick match -> Settle + Refund |
| Ledger Math              | Available + Used + Reserved = Total Account Capital    |
| Holdings vs Positions    | CNC Delivery = Holdings, MIS Intraday = Positions      |
| Real-Time Ticks          | TrueData WS -> in-memory Map -> limit triggers + Sockets|
| AI Assistant             | Parallel DB Context -> Gemini 2.5 Flash -> ChatModel   |
| Tournaments              | Independent Virtual Portfolio -> Live ROI Leaderboard  |
| Security                 | Joi validation, bcrypt (salt 10), tokenVersion logout |
└──────────────────────────┴──────────────────────────────────────────────┘
```

16.4 Final End-to-End Feature Summary

Zerodha Clone represents a fully cohesive, full-stack trading application engineering effort. By faithfully modeling real-world order execution conditions, multi-tier balances, background tick evaluation, isolated tournament competitions, and contextual financial AI intelligence, the platform bridges the gap between basic web tutorials and real institutional-grade fintech software.